

Reading APFS
Notes Toward a Fast Index of an Apple File System Volume

apfs-fastindex project

2026-05-19

Contents

Preface	v
I The Problem	1
1 Why APFS Resists MFT-Style Indexing	2
1.1 What APFS Provides Instead	2
1.2 The Right Question	3
1.3 A Narrow Supported Core	3
1.4 Why Live Latest Cannot Be The Default	4
1.5 What This Manual Does Not Promise	4
2 Method	5
2.1 Claim Discipline	5
2.2 Oracle Plurality	5
2.3 Paired Probes	6
2.4 The Smallest Useful Fixture	6
2.5 Fail Closed By Default	7
2.6 The Style Of A Finding	7
II The Container and the Object Map	8
3 The Container	9
3.1 Block Zero As Locator	9
3.2 Selecting The Checkpoint	9
3.2.1 Why The Highest Valid XID	10
3.2.2 Live State Is Not Pinned	10
3.3 The Checkpoint Map Ring	11
3.4 Roots Reachable From The Selected Checkpoint	11
4 The Object Map	12
4.1 Lookup By (OID, Max-XID), Not By OID Alone	12
4.2 Hard Stops At OMAP Open	12
4.3 Hard Stops At OMAP Lookup	13
4.4 An Empirical Correction: The B-Tree TOC Layout	13
4.5 Cross-Tool Validation Of The Lookup Contract	14
4.6 What The OMAP Does Not Do	14

III	The File-System Tree	15
5	The File-System Tree	16
5.1	The Finding, Stated Once	16
5.2	Why This Is The Easy Mistake	16
5.3	The Probe That Found It	16
5.4	The Object-Header Validation Detail	17
5.5	Required Record Families	18
5.6	Topology Summary	18
6	Record Bodies and the Xfield Cursor	20
6.1	The Cursor Rule	20
6.2	Why The Reference Reads Otherwise	21
6.3	Evidence From The Proof Fixture	21
6.4	The Required Xfield Types	22
6.5	Field-Level Cross-Tool Equality	23
6.6	When To Use This Chapter	23
7	The Fail-Closed Boundary	24
7.1	Where The Boundary Sits	24
7.2	The Per-Record Catalogue	25
7.2.1	Short bodies	25
7.2.2	Malformed names	25
7.2.3	Malformed xfields	25
7.2.4	Malformed xattrs	25
7.3	The Cross-Record Cases	26
7.4	Why The Boundary Is Tighter Than It Looks	26
7.5	The Object-Header Side	26
7.6	The Boundary And The Fallback	27
IV	Names and Sizes	28
8	Sizes and Their Precedences	29
8.1	Logical Size	29
8.1.1	The Precedence Rule	29
8.1.2	The Load-Bearing Case: Compression	31
8.1.3	The Hint That Is Not A Size	31
8.1.4	Per-Inode Results	31
8.2	Allocated Size	32
8.2.1	The Precedence Rule	32
8.2.2	Why <code>allocated_size</code> Is The Only Candidate	32
8.2.3	The Sparse-File Lift (EX-26 Hypothesis A)	33
8.2.4	The decmpfs Lift (EX-26 Hypothesis F)	33
8.2.5	Per-Inode Results	34
8.3	Directory Aggregates	34
8.4	What This Chapter Does Not Promise	35

9	Names Across Volume Modes	36
9.1	The Finding	36
9.2	Two Volume Modes Exist	36
9.3	What The Modes Do Affect	36
9.3.1	The Lookup Oracle	37
9.3.2	Rust Vs POSIX Equality	37
9.4	The Deliberate Non-Promise	37
9.5	Names And Aggregates	38
9.6	What Future Work Will Need	38
V	Boundaries	39
10	Identity, Reuse, and What a Cache Key Cannot Be	40
10.1	The Tempting Mistake	40
10.2	The Shape Of A Safe Cache Key	41
10.3	What EX-07 Proved And What It Did Not	41
10.4	The Identity-Tracking Discipline	42
10.5	What This Chapter Forecloses	42
11	Support, Readability, and the Fallback	43
11.1	Four Properties, In Order	43
11.2	Readable Is Not Supported	43
11.3	The Current Allowlist	44
11.4	EX-28 Live-Volume Raw Mode (Hypothesis C: Blocked by the Kernel)	44
11.5	EX-29 Local-Snapshot Enumeration (No User Snapshots on This Host)	45
11.6	Fail-Closed Triggers	46
11.7	The Fallback	46
11.8	Resilience In The Fallback	47
11.9	Parallel Fallback	47
11.10	Firmlink Overlay Dedup	48
11.11	Auto-Dispatch	49
11.12	Why The Boundary Is Permanent	49
VI	Engineering	50
12	Measurement	51
12.1	The Standing Baseline	51
12.2	The Micro-Optimisation Pass	52
12.3	The Performance-Lever Map	54
12.3.1	The Parallel Slice In Product	55
12.4	The Structural-Density Pass	56
12.5	What The Numbers Do Not Say	57
12.6	The Measurement Discipline	58

13 Open Questions	59
13.1 Allocated Size	59
13.2 Snapshot-Assisted Scanning	59
13.3 Scanner Ergonomics and Performance	60
13.3.1 The Native Shell	61
13.3.2 The Treemap Layout	63
13.4 Deliberately Deferred	65
13.5 The Manual's Own Future	66
Glossary	67
Appendix: Experiment Register	72

Preface

This manual is a record of what the `apfs-fastindex` project has learned about the Apple File System on disk and about the specific problem of enumerating a single volume quickly and correctly. It is not a guide to the repository. The repository is a working environment; this manual is the textbook the working environment built up.

The reader the manual is written for is a careful engineer who wants to read APFS metadata directly. We assume familiarity with copy-on-write filesystems and with B-trees, but not with APFS specifically. Every claim that is not common knowledge is sourced or labelled.

Claim Discipline

Three labels appear throughout the text.

Spec. Backed by Apple’s public reference¹ or by another public, durable specification.

Observation. Backed by a controlled experiment in this project or by behavior that converges across multiple independent open-source readers (`linux-apfs-rw`, `apfs-fuse`, `dissect.apfs`, `go-apfs`, `libfsapfs`, the Sleuth Kit’s APFS module).

Hypothesis. Plausible, useful for design, not yet proven. Anything labelled Hypothesis does not justify a product claim.

Where Spec and Observation disagree, Observation wins. Apple’s reference is one citation among many; converged code is often more decisive about what bytes actually mean. Where the disagreement matters, the text says so and names the smallest probe that would settle it.

Oracle Plurality

There is no single oracle for an APFS indexer. Each feature has its own:

- Namespace: POSIX traversal of the same volume at the same selected state.
- Logical size: `st_size` per inode (or symlink-target byte length for symlinks).
- FS-record body: cross-tool field equality between independent decoders against the same raw container.
- Allocated size: `st_blocks * 512`, for cases that have been explicitly validated.

¹*Apple File System Reference*, retrieved 2026-05-13, <https://developer.apple.com/support/downloads/Apple-File-System-Reference.pdf>.

- Incremental correctness: a fresh full reparse of the same selected state.
- Boot-root semantics: the user-visible macOS namespace, and only when the product mode explicitly targets it.

Every result in this manual names the oracle that verified it.

Fail Closed By Default

The parser refuses to guess. Outside the tested allowlist, it stops with a typed error and the caller is responsible for choosing a fallback. Plausible but unproven is treated as "do not emit." The discipline is not paranoia: it is the cheapest way to keep silently wrong rows out of aggregates and out of the user interface.

What Is And Is Not Here

The manual presents findings about APFS as a textbook would: each chapter takes one face of the system, states what is known, gives the evidence, and notes what remains open. Where the project's implementation makes a non-obvious choice, the choice is explained from first principles rather than from code organisation.

What is not here: a tour of the repository, a list of files, a list of commits, a list of experiments treated as work units rather than as evidence. Those live in the repository itself; the manual cites them only where citation supports a claim.

A Note On The Voice

We have tried to write as though the reader will need to defend each claim to a sceptic. That style is at odds with the way reverse engineering is usually written up, but the project has been disciplined about evidence, and the manual should reflect that. A sentence that cannot be defended should not have been written.

Part I

The Problem

Chapter 1

Why APFS Resists MFT-Style Indexing

WizTree owes its speed to a peculiarity of NTFS: the Master File Table. The MFT is a single, flat, contiguous structure in which each file is one record. Indexing an NTFS volume reduces to reading that table sequentially. Subdirectory recursion, repeated stat calls, and path-by-path traversal are all unnecessary.

APFS provides nothing of the kind, by design.

1.1 What APFS Provides Instead

An APFS container is a copy-on-write region of transactional B-trees. There is no flat metadata table to read; there are at least three trees the indexer must touch for every file, in a specific order, all under a single chosen transaction state.

Spec. A container holds:

- a stable block-zero locator that points at the *checkpoint descriptor area*,
- the descriptor area itself, which carries a ring of container superblocks — one per transaction — each of which is a self-consistent view of the container at that transaction,
- the *container object map* (OMAP), a B-tree that resolves virtual object identifiers to physical block addresses under a chosen transaction,
- one or more *volume superblocks*, reached through the container OMAP,
- per-volume OMAPs that resolve the volume’s virtual objects,
- per-volume *file-system trees*, B-trees of variable-length records keyed by `(object_id, type)` that hold directory entries, inodes, extended attributes, hard-link records, and data stream metadata.

Every metadata read is therefore at least three indirections: container, volume, FS tree. The lookups themselves are cheap, but each of them needs the correct OMAP and the correct transaction context. The naive question — *where is the MFT?* — has no answer.

1.2 The Right Question

The right question is structural:

What is the minimum set of structures we must traverse, in what order, under what state pin, to correctly enumerate one volume?

The answer fills most of this manual. In compressed form:

1. Locate the descriptor area from block zero.
2. Select one checkpoint from that area — the highest-XID checksum-valid container superblock.
3. Walk the checkpoint map; resolve the container OMAP.
4. Look up the target volume superblock through the container OMAP.
5. Open the volume OMAP; look up the FS-tree root.
6. Walk the FS tree, resolving internal-node child references through the volume OMAP at every level.
7. Decode the FS-tree leaf records into directory entries, inodes, xattrs, sibling links, and dstreams.
8. Reconstruct paths, file identity, and logical size from those records.

Each step is independently load-bearing. Skip the OMAP lookups at the FS-tree internal level (chapter 5) and you read random unallocated blocks. Choose the wrong checkpoint (chapter 3) and you read an incoherent transaction. Mis-decode an xfield blob (chapter 6) and you silently emit wrong sizes.

1.3 A Narrow Supported Core

The manual’s scope is deliberately narrow:

- one APFS volume,
- at one pinned transaction state,
- correct namespace,
- per-file logical size, with per-file allocated size where an on-disk field has been oracle-validated,
- fail closed outside the tested allowlist.

The scope is narrow because the cost of being approximately right is unbounded: aggregates derived from silently wrong rows propagate into treemaps, into “where did my disk space go” decisions, and into filesystem operations the user takes based on the result. The cost of being narrowly right and failing closed everywhere else is bounded: the fallback path produces a correct answer through supported APIs, more slowly.

This is not a hedge. It is the way to read APFS metadata at all without silently corrupting downstream totals.

1.4 Why Live Latest Cannot Be The Default

A particular instinct deserves to be killed early. The instinct says: "just read the latest checkpoint each time; the volume will be self-consistent enough." It is wrong on mounted volumes that the user is using.

Observation. On a mounted lab image, writes from a concurrent process advance the visible latest checkpoint underneath the reader's eyes. Reading "latest" twice may return two different transactions; mixing objects across them produces an incoherent namespace. The result has no oracle to compare against because no oracle ever saw the mix. The probe that demonstrated this (EX-05) is the load-bearing counterexample for the design.

Live latest is therefore not a supportable mode. The supported raw mode requires a state that can be pinned and held: a detached image, an explicitly stable APFS source, or a tightly controlled lab volume. Outside that allowlist the indexer falls back to a supported operating-system API.

1.5 What This Manual Does Not Promise

Several APFS surfaces are partially or wholly outside the manual's scope, not because they are unimportant but because they have not yet been validated end to end. They appear later, in chapter 13, as open questions:

- exclusive and shared bytes per inode, snapshot-retained bytes, and any size column that requires the extent-reference tree to be joined with per-inode extents;
- persistent incremental caches that reuse prior subtree summaries (chapter 10 develops the identity rules that any such cache would need);
- the Finder-visible “/” namespace assembled from system and data volumes, firmlinks, and snapshots on a modern macOS startup disk (chapter 11 documents the conservative firmlink-overlay dedup the fallback walker performs; the merged Finder semantics proper remain unclaimed).

A claim from this manual extends only to the things the manual explicitly proves.

Chapter 2

Method

The findings in this manual were arrived at through a deliberately restrictive method. The method is worth stating explicitly because it shapes which claims appear, and because the same discipline is what allows the reader to trust those claims.

2.1 Claim Discipline

Three categories. They are never blurred.

Spec. A claim backed by Apple’s public reference or by another durable specification. Spec claims are useful but not sufficient. The reference describes intent; what is on disk is what the running system writes, and the two occasionally disagree.

Observation. A claim backed either by a controlled experiment on a real APFS image, or by behaviour that converges across multiple independent open-source readers. Convergence across six readers (`linux-apfs-rw`, `apfs-fuse`, `dissect.apfs`, `go-apfs`, `libfsapfs`, the Sleuth Kit’s APFS module) plus the reference is harder to fault than the reference alone. For performance claims the same convergence pattern applies across independent fast macOS scanners (`dumac`, `macdirstat`, `jwalk`) and the xnu kernel source itself.

Hypothesis. A claim that is plausible and useful for design but has no probe or convergence behind it yet. Hypothesis does not license product behaviour. If the parser would emit a row only on a Hypothesis-class claim, the parser does not emit the row.

The two rules that follow from these labels.

Where Spec and Observation disagree, Observation wins. Section 6 on the xfield cursor rule is one example; section 5 on the FS-tree internal edge is another. In both cases the reference is easy to read in a direction that the converged code does not support. The converged code is right.

Where a Hypothesis matters, the smallest decisive probe is named. The manual does not leave open questions floating; it leaves them attached to the experiment that would close them.

2.2 Oracle Plurality

There is no single oracle for an APFS indexer. The phrase "the oracle" is a tell that someone has glued mismatched validation sources together and called the result authoritative. Each feature owns its own oracle.

Namespace. POSIX traversal (`os.walk`) of the chosen volume at the chosen state. Path bytes, entry type, hard-link grouping by inode, symlink target.

Logical size. `st_size` per inode; for symlinks, the target byte length returned by `readlink`. This is the public, stable per-file size that user-space tools agree on.

FS-record body. Cross-tool field equality between independent decoders against the same raw container. The `go-apfs identitydump` tool plus an independent Python decoder against the same `/dev/rdiskN` is the form the project uses.

Allocated size. `st_blocks * 512`, and only on cases that have an explicit validated mapping from raw fields to it.

Incremental correctness. A fresh full reparse of the same selected state. Any cache reuse must equal this.

Boot-root semantics. The user-visible macOS namespace — System/Data join, firmlinks, current snapshot — and only when the product mode targets it.

A probe that does not name its oracle does not produce evidence.

2.3 Paired Probes

The most informative experiments build the fixture, capture every oracle, run the candidate parser, and compare — all in one execution of one script. This rules out the kind of subtle drift that breaks an experiment: a fixture rebuilt with slightly different operations, a volume remounted between probes, a parser pointed at a different container than the oracle.

Two patterns in particular keep recurring.

Detach-and-reattach. Build the fixture mounted; capture the mounted POSIX oracle; `hdiutil detach`; `hdiutil attach -nomount -readonly`; run the raw parser; diff. The detach fixes the visible checkpoint, so the raw parser and the mounted oracle agree on what state to compare.

Cross-tool agreement. Run the candidate parser and an independent third-party reader against the same raw container in the same probe. Record the parser’s resolved (`oid`, `paddr`, `xid`) samples; replay the object-header validation rules at every resolved `paddr`; confirm the two tools agree on the FS-tree root OID. A divergence in *which* checkpoint each tool selected is itself recorded as a caveat, not silenced.

2.4 The Smallest Useful Fixture

Probes prefer the smallest fixture that exercises the question, not the largest fixture that might. The proof fixture used through most of this manual is intentionally tiny: two directories, a renamed file, a moved file, a hard link, a sparse 1 MiB file, a clone, an append after the clone, and a symlink. Roughly seven user-visible entries.

That fixture is enough to surface:

- the FS-tree internal-OID-vs-paddr distinction (when the tree grows past one leaf node),

- the xfield cursor rule (the sparse inode has three xfields, one of which has a non-multiple-of-eight `x_size`),
- the logical-size precedence over ordinary, sparse, clone, hard-linked, and symlink rows,
- the unique-inode aggregate policy (hard links must not double-count).

Larger fixtures appear only when a specific question requires a feature the small fixture cannot express, such as the case-insensitive vs case-sensitive volume comparison in chapter 9 or the compressed-file precedence test in chapter 8.

2.5 Fail Closed By Default

A parser that emits rows for cases it does not understand is worse than one that emits nothing. The user can recognise a missing row and look elsewhere; a wrong row propagates into a directory aggregate, into a treemap, and into the decisions the user makes from the treemap. Fail-closed is not an aesthetic preference. It is the only way to bound the cost of being wrong.

Every step in the parser is wrapped in a typed validation gate. The single chokepoint for object-header validation is one function (`validate_object_block` in the Rust crate); every reader calls it before trusting any block. The single chokepoint for record-body validation is one function (`decode_fs_record`); every emission path calls it before trusting any record. Each gate returns a typed error with the rule that failed.

Outside the allowlist, the parser does not partially succeed. It does not "try its best." It returns a reason and the fallback path (chapter 11) takes over.

2.6 The Style Of A Finding

The manual presents a finding the same way each time: a one-line statement, the evidence, the implementation consequence, and the limits of what was checked. The structure is borrowed from how working proofs are written: the claim must survive standing alone, but the proof must be present.

Part II

The Container and the Object Map

Chapter 3

The Container

Block zero is the locator. It is not the authoritative container superblock. Treating block zero as the source of truth is the first mistake an APFS reader can make; the second is reading the highest checkpoint without validating it; the third is reading the latest checkpoint while the volume is changing. This chapter is concerned with avoiding all three.

3.1 Block Zero As Locator

APFS is copy-on-write at the container level. Each transaction writes a new container superblock (`nx_superblock_t`, henceforth NXSB) into the checkpoint descriptor area. Block zero is the stable known location at which a reader can find the descriptor area. It is updated lazily; the authoritative live NXSB is one of the copies in the descriptor ring.

The pieces of block zero a reader needs:

- `block_size` at offset `0x24`: validated to be a power of two within `[4096, 65536]`.
- `nx_xp_desc_blocks` at offset `0x68`: the length of the descriptor area in blocks.
- `nx_xp_desc_base` at offset `0x70`: the first block of the descriptor area. The high bit signals a *non-contiguous* layout, which this work does not support and which triggers a typed hard stop. A non-contiguous descriptor area would require a separate checkpoint-mapping-tree implementation.

Block zero is itself an APFS object and must validate as one: NXSB magic at offset `0x20`, object type `NX_SUPERBLOCK`, object id equal to the constant NX-superblock OID (1), and a valid Fletcher-64 checksum. A block zero that does not satisfy these is not an APFS container.

Observation. The `block_size` and descriptor base in block zero must agree with the selected NXSB's view of the same fields. A discrepancy means block zero is stale relative to a descriptor write that did not (yet) propagate. The parser rejects the source rather than guessing which view is correct.

3.2 Selecting The Checkpoint

The descriptor area is a contiguous run of `nx_xp_desc_blocks` blocks. Some are checkpoint maps (chapter 3.3); some are NXSB candidates. The selection rule:

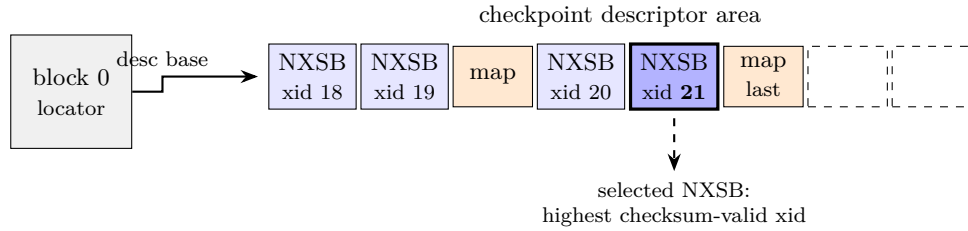


Figure 3.1: Block zero locates the descriptor ring; the selected NXSB is the highest checksum-valid candidate in the ring. Several earlier NXSBs may remain valid; **map** blocks carry checkpoint-map contents; trailing slots may be unwritten between commits.

1. Iterate every block in the descriptor range.
2. For each, check the NXSB magic at offset 0x20.
3. If the magic matches, validate object type (`NX_SUPERBLOCK`) and Fletcher-64 checksum.
4. Record the candidate.
5. After the scan, pick the candidate with the highest XID.

The highest valid XID is the selected NXSB. Its XID becomes the *scan XID*, and every subsequent object resolution for the run is performed at this XID.

Blocks that carry NXSB magic but fail object-type or checksum validation are recorded as skipped descriptors with the reason. They are not silently ignored; a future support broadening must justify why they were skipped.

Observation. Three or four NXSB candidates is typical for a recently written container. The first execution of EX-15 against the EX-14 fixture saw four candidates with XIDs 18–21; all four passed strict validation. Multiple candidates is not a sign of corruption. Zero candidates is.

3.2.1 Why The Highest Valid XID

The candidates are not chronologically ordered in the descriptor area; they are written into a ring. The XID is the only monotonic identity for transaction order. The highest checksum-valid XID is the most recent transaction the writer completed. A higher XID with an invalid checksum is a transaction the writer started but did not finish; the parser treats those as failed and ignores them, with the result that selection is always self-consistent.

The rule has a corollary: a parser that picks a lower XID when the highest is valid has chosen a stale state for no reason and will disagree with anything else that picks the latest.

3.2.2 Live State Is Not Pinned

Observation. On a mounted APFS volume that other processes are using, the highest valid XID moves under the reader. Two reads of "latest" may return two different transactions. Mixing objects from different transactions produces an incoherent namespace. EX-01's churn probe demonstrated this on a mounted lab image.

The fix is structural rather than algorithmic: raw mode only runs on sources where state can be pinned. Detaching an image fixes the visible state. So does an explicit caller commitment that a device's checkpoint will not advance. Live mounted volumes are readable-but-not-supported; chapter 11 develops the vocabulary.

3.3 The Checkpoint Map Ring

A selected NXSB references a chain of checkpoint maps (`checkpoint_map_phys_t`) in the descriptor area. The map chain carries the ephemeral object identifier table for the selected checkpoint: the OIDs and physical addresses of objects that live in memory between transactions and are written into the checkpoint data area at commit. The reader needs the map to resolve those ephemerals.

The walk rule:

1. Start at `nx_xp_desc_index` in the selected NXSB.
2. Read the block; validate as an APFS object with type `OBJECT_TYPE_CHECKPOINT_MAP`.
3. Record the (`oid`, `paddr`) mappings.
4. Advance through subsequent map blocks in the descriptor area.
5. Stop when a block carries the `CHECKPOINT_MAP_LAST` flag.

A chain that runs out of length before the last flag is malformed. The parser fails closed.

Empirical correction. The Apple reference describes ephemeral objects in a way that reads naturally as "the checkpoint map block itself is ephemeral." It is not. Validation against a real container demonstrates that `OBJECT_TYPE_CHECKPOINT_MAP` blocks carry the `OBJ_PHYSICAL` storage flag, not `OBJ_EPHEMERAL`. The validator must expect `Physical` storage for these blocks. This correction was landed during EX-11 as a one-line constant change with a synthetic malformed-checkpoint-map regression test.

3.4 Roots Reachable From The Selected Checkpoint

The selected NXSB names five things the reader needs:

1. `nx_omap_oid`: the OID of the container OMAP.
2. `nx_fs_oid[]`: an array of volume superblock OIDs.
3. Feature, incompatible-feature, and read-only-compatible feature masks.
4. Reaper and space-manager OIDs (not used by an indexer; their presence is required, their content is not).
5. Container UUID (used for source identity in caches and diagnostics).

The incompatible-feature mask is the gate for format drift (chapter 11). Any bit the reader does not recognise stops the run. Better to refuse a container with a new feature bit than to silently emit a row that depends on the bit's meaning.

The container OMAP and the volume superblocks both require object resolution, and that is the next chapter's subject. The point to fix here is that block zero, the descriptor area, and the checkpoint map together establish a single coherent transaction context. Everything that follows runs inside that context. The context is not negotiable.

Chapter 4

The Object Map

APFS resolves logical object identifiers to physical block addresses through a B-tree called an object map, or OMAP. There is one OMAP per container (resolving volume superblocks and ephemerals) and one per volume (resolving the FS-tree root and its descendants). The OMAP is the single mechanism by which a copy-on-write filesystem can have stable object identity over time.

The chapter is short, but its claim is foundational: the OMAP is a *two-key* lookup, and the second key is non-negotiable.

4.1 Lookup By (OID, Max-XID), Not By OID Alone

Spec. An OMAP key is (`oid`, `xid`). An OMAP value names a physical address, a size, and a flag word. The OMAP can hold multiple entries for a single OID, each at a different XID, corresponding to different versions of the object over time.

Spec. An OMAP lookup specifies an `oid` and a *max XID*. The B-tree returns the entry with the highest stored XID that is *less than or equal to* the requested max XID. The semantics are lower-bound rather than exact match: the lookup is asking "give me the version of `oid` that was current at or before this scan."

The implication for the caller is structural. The resolver input is not a bare OID. It is a triple:

1. the *OMAP context* — which OMAP owns the OID,
2. the OID itself,
3. the scan XID.

A caller that hands the resolver only an OID is asking the wrong question. The OID by itself does not determine which version of the object is meant; the scan XID does.

The scan XID is the same XID for the entire run: the one selected from the descriptor area in chapter 3. Mixing scan XIDs across object resolutions within one run reintroduces the incoherence that pinning the checkpoint was supposed to prevent.

4.2 Hard Stops At OMAP Open

When the parser opens an OMAP — reads the `omap_phys_t` root object and validates it — the `omap_phys.flags` field gates whether the OMAP can be used at all.

OMAP_PHYS_ENCRYPTING.

OMAP_PHYS_DECRYPTING.

OMAP_PHYS_KEYROLLING.

OMAP_PHYS_CRYPTO_GENERATION_FLAG.

Any of these set means an encryption operation is in flight on the container or the volume. Raw mode cannot be supported in such a state: the bytes the reader sees do not yet match the bytes the running system will commit. The parser fails closed; the caller falls back to a supported API.

Observation. Any *unknown* bit in `omap_phys.flags` is also a hard stop. APFS reserves bits over time; an unknown bit means the running OS knows something the parser does not, and the parser must not infer behaviour. This is the format-drift discipline of chapter 11 applied at the smallest object that hosts it.

4.3 Hard Stops At OMAP Lookup

Each value entry returned by an OMAP lookup carries its own flag word. The flags decide whether the value is usable.

OMAP_VAL_DELETED. Negative result, not an error. The OID had a mapping at some earlier XID but is no longer visible at the scan XID. The caller interprets this as "the object is not present" and proceeds accordingly. The parser does not hard-stop.

OMAP_VAL_ENCRYPTED. The value is per-object encrypted. Raw mode cannot decrypt; hard stop.

OMAP_VAL_NOHEADER. The target block does not carry an `obj_phys_t` header at all. A structural assumption the rest of the parser depends on has been broken; hard stop.

OMAP_VAL_CRYPTO_GENERATION. Encryption-generation mismatch between the OMAP entry and the volume; hard stop.

unknown bits. Format drift; hard stop.

The discipline is tighter at lookup time than at open time only because lookup is also where the parser sees per-entry encryption behaviour. The principle is the same: known flag means known behaviour; unknown flag means the parser does not understand the running system and refuses to guess.

4.4 An Empirical Correction: The B-Tree TOC Layout

The OMAP is a B-tree, and B-tree leaf nodes in APFS carry a key/value table-of-contents that the reader must traverse to find values. The TOC format is uniformly described across the Apple reference and the converged code, but a single offset interpretation gates whether the reader actually decodes the values correctly. The project landed this correction during EX-12.

Observation. The TOC `v_off` field measures the distance from the *end* of the value area to the *start* of the value. The value then runs *forward* from that start position for `fixed_value_size` bytes (or `v_len` for variable-length values).

The naive reading is "the value extends backward from `v_off` toward the start of the node." Under that reading, OMAP values decode with a bizarre flag word — specifically `flags=0x10ffff` on the proof fixture — and the rest of the parser breaks downstream.

The corrected reading produces clean OMAP values (`flags=0x0`, `size=4096`, `paddr=...`) and matches every converged reader. The fix is one constant change in B-tree value extraction; the regression is one synthetic OMAP test. The cost of the bug is high (the parser is silently wrong about every lookup) and the test that catches it is cheap, so the project keeps the synthetic OMAP test in the regression suite.

4.5 Cross-Tool Validation Of The Lookup Contract

EX-12 paired the project's parser with `go-apfs identitydump` against the same `/dev/rdiskN`. The probe:

1. built a fresh proof fixture,
2. ran the project's parser, recording every resolved (`oid`, `paddr`, `xid`) sample published by the OMAP,
3. replayed object-header validation rules in Python at each published `paddr`,
4. ran `go-apfs identitydump` against the same raw container,
5. compared the two tools' agreement on the FS-tree root `oid`.

The two agreed on the FS-tree root OID. They did not agree on which scan XID to select: the project's parser picked the higher XID visible in the descriptor ring, `go-apfs` appeared to pick one lower. That divergence is recorded as a caveat (`go_apfs_active_state_observation`) and not silenced; it does not affect the value of the cross-tool oracle, because both tools' OMAP lookups internally use a consistent scan XID and produce internally consistent root resolutions.

The lesson generalises. Two tools that disagree on selection but agree on the OID they reach through that selection are still useful as oracles for each other, as long as the parser records the selection difference. A cross-tool oracle that silences a known disagreement is worse than no oracle at all.

4.6 What The OMAP Does Not Do

The OMAP does not resolve objects *within* the FS tree. That is the source of the next chapter's finding, and the trap that cost the project a fixture.

FS-tree internal-node entries also carry OIDs, and those OIDs also require OMAP resolution at the scan XID — specifically, through the *volume* OMAP, not the container OMAP. The OMAP machinery from this chapter is exactly what is needed; what changes is the caller. The next chapter develops the rule and its empirical evidence.

Part III

The File-System Tree

Chapter 5

The File-System Tree

The chapter contains the single most consequential reverse-engineering finding in the project so far. The finding is small; its absence cost a fixture and almost cost the project's first defended scope; its presence makes the parser correct on every FS tree dense enough to grow past one leaf node.

5.1 The Finding, Stated Once

Observation. An FS-tree internal-node entry has an 8-byte value. That value is a *virtual object identifier*, not a physical block address. To read the child node, the walker must resolve the value through the *volume* OMAP at the scan XID. A walker that reads the value as a paddr will, on any FS tree of depth ≥ 1 , read random blocks.

This is the rule. The rest of the chapter develops the consequences and the empirical evidence.

5.2 Why This Is The Easy Mistake

The Apple reference documents that FS-tree objects live in the volume's virtual address space. It does not foreground that *internal-node child references are members of the same address space*. The reference reads naturally as "the internal value is a pointer into the same area as physical extents," and many B-tree formats indeed encode internal child references as direct physical offsets.

The converged open-source code is unanimous in the right direction, but the unanimity is implicit. Each reader hands its B-tree walker an "object resolver" that internally does the OMAP lookup, and the walker code does not need to think about it. A reader who reimplements the walker without an equivalent resolver — which is what the project's first Rust walker did — will not notice the lapse on a small fixture, because a small enough FS tree fits in a single leaf node and has no internal entries to follow.

The project missed this through the EX-10, EX-11, and EX-12 fixtures. Each was small enough that the FS tree had no internal level, and the walker was never asked to dereference an internal value. EX-14 finally produced a fixture dense enough to need an internal level, and the parser broke.

5.3 The Probe That Found It

Observation. EX-14 reported APFS object validation failed: checksum mismatch at block 1031 and aborted before publishing a selected checkpoint. The probe `fsck_apfs -n`

returned clean; `go-apfs identitydump` succeeded on the same container. The image was not malformed.

EX-15 replayed the upstream context in Python, validating every candidate against the strict rules of chapters 3 and 4. Every NXSB candidate validated. Every checkpoint map block validated. The container OMAP, volume superblock, and volume OMAP all validated. The FS-tree root validated. The walker then dereferenced an internal entry whose 8-byte value was 1031, read block 1031, and encountered 4096 bytes of zero — an unallocated block — whose Fletcher-64 mismatched.

The volume OMAP, queried for `oid=1031` at the scan XID, returned `paddr=505`. Block 505 was a valid FS-tree internal node. The walker had been asked to follow an OID, and had instead read the OID’s numeric value as a paddr.

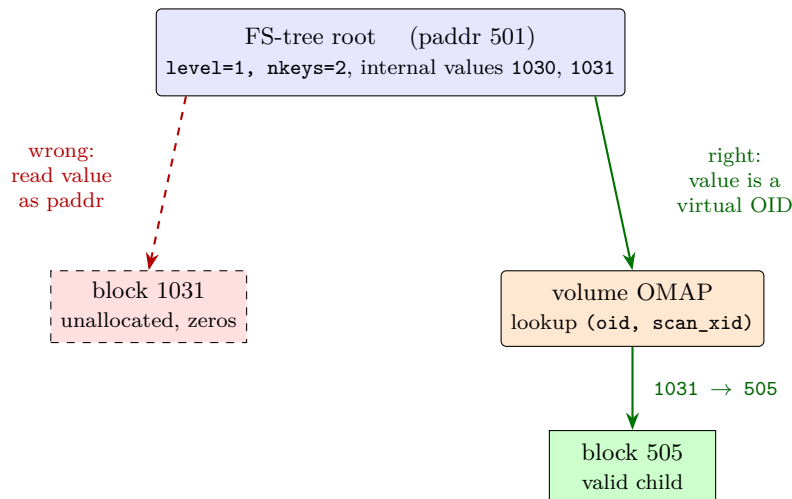


Figure 5.1: The headline finding. An FS-tree internal-node value is a virtual OID; the walker must consult the volume OMAP at the scan XID before reading the child. Reading the value as a paddr lands on an unallocated block.

The fix is structural. `walk_fs_node` now takes the volume OMAP resolver and performs an `(oid, scan_xid)` lookup for every internal entry before reading the child. A missing OMAP mapping returns a typed `InvalidObject` hard stop naming the OID.

Two synthetic regression tests in the Rust crate close the bug:

1. `fs_tree_internal_value_is_virtual_oid_resolved_via_omap` builds an 8-block synthetic image where the FS-tree root’s internal entry has value 1030. With the fix, the walker follows OMAP to the correct paddr and reads the leaf record. Without the fix, the walker short-reads on block 1030.
2. `fs_tree_internal_oid_missing_from_omap_is_hard_stop` asserts that an internal value pointing at an OID with no OMAP mapping returns a typed error mentioning the missing OID rather than silently skipping or attempting a paddr fallback.

These tests now run on every commit; a future regression that reintroduces the bug would be caught at the synthetic level rather than at the next fixture variant.

5.4 The Object-Header Validation Detail

Two further details follow from the FS-tree root being virtual.

Observation. The FS-tree root is reached through the volume OMAP, so on disk it carries a virtual `obj_phys_t` header. The storage flags are `0x0000_0000` (virtual), not `OBJ_PHYSICAL`.

Observation. A virtual object’s `o_oid` field does not equal the block’s physical address; the OID is the virtual identity, the `paddr` is independent. The validator therefore must *not* require `o_oid == paddr` for virtual objects. The right check is `o_oid == omap_lookup.oid`: the OID the walker resolved through the OMAP must be the OID the on-disk header advertises.

The corollary is that the project’s object-header validator has two modes: `ExpectedStorage::Physical`, which requires `o_oid == paddr`, and `ExpectedStorage::Virtual`, which does not. Every reader picks one at the call site. The fail-closed rule is preserved: a block validated under the wrong mode is rejected.

5.5 Required Record Families

An indexer that reproduces POSIX namespace plus logical and allocated size needs five families of FS-tree records and one xfield family.

DIR_REC. Directory entries. Key carries the parent inode’s file ID and the name hash; value carries the child file ID and entry type. The directory structure is built by walking `DIR_REC` records.

INODE. File, directory, symlink, and other entry metadata. Carries the parent ID, link count, mode, internal flags, and an xfield tail.

XATTR. Extended attributes. The indexer cares about three specifically: `com.apple.fs.symlink` (the symlink target), `com.apple.decmpfs` (the compression metadata header), and `com.apple.ResourceFork` (out of the defended scope but tracked for the decmpfs fail-closed gap).

SIBLING_LINK, SIBLING_MAP. Hard-link unification. A directory entry that participates in a hard-link group carries a sibling ID xfield; `SIBLING_MAP` closes the mapping from sibling ID to file ID.

dstream fields. Logical size sources. Carried either inside `INODE`’s xfield tail as `INO_EXT_TYPE_DSTREAM` or as a dedicated `j_dstream_id_val_t` record.

Other families are decoded but not used for the indexer’s emission rows: `file_extent` (chapter 13’s extent-reference territory), `snapshot_metadata` and `snapshot_name` (deferred), and an “unsupported” bucket for record types the parser does not recognise. A record falling into the unsupported bucket on a volume that is otherwise in the allowlist triggers a typed validation gap that the caller must consume.

5.6 Topology Summary

To enumerate one volume, the walker performs:

1. Container OMAP lookup at scan XID for the volume’s superblock OID, returning the volume superblock `paddr`.
2. Volume superblock decode: the feature allowlist is enforced; unknown incompatible bits, encryption, sealed-volume bits, and normalization-insensitive bits force `Unsupported(reason)`.

3. Volume OMAP open; same encryption / unknown-flag gates as chapter 4.
4. Volume OMAP lookup at scan XID for the FS-tree root virtual OID, returning the root paddr.
5. FS-tree root validation as a virtual B-tree node.
6. Recursive descent: at each internal node, an OMAP lookup per child entry; at each leaf node, decode of every record per the families above.

Every reader in the project's Rust crate that touches an FS-tree node goes through this path. There are no shortcuts; in particular there is no path that treats an internal value as a paddr.

The next chapter handles what is inside the leaf records: the variable-length tails, and the single source-backed rule for reading them.

Chapter 6

Record Bodies and the Xfield Cursor

An FS-tree leaf record has a fixed-size header and a variable-length tail of extended fields. The fixed header is easy. The variable tail is subtle: Apple’s reference admits an ambiguous reading, the converged code is unanimous in a single direction, and the right rule must be applied uniformly across every record family that uses xfields.

6.1 The Cursor Rule

Observation. The xfield tail follows one cursor rule, derived from `linux-apfs-rw`, `apfs-fuse`, `dissect.apfs`, `go-apfs`, `libfsapfs`, and the Sleuth Kit:

1. The tail begins with `xf_blob_t`: 4 bytes containing `xf_num_exts` and `xf_used_data`.
2. Immediately after is the metadata table: `xf_num_exts` entries of `x_field_t`, each 4 bytes (`x_type`, `x_flags`, `x_size`).
3. The value area begins immediately after the metadata table. There is no extra alignment of the value-area start.
4. Reading in metadata-table order, each value consumes `round_up(x_size, 8)` bytes. The cursor advances by the padded length, not by `x_size`.
5. `xf_used_data` equals `sum(round_up(x_size, 8))` for the record’s xfields.

That is the rule. It is one rule across all xfield-bearing record families, and a record whose `xf_used_data` disagrees with the padded sum is malformed.

The rule fits in a few lines of code:

```
1 let mut cursor = metadata_end;
2 let mut padded_total = 0usize;
3 for entry in metadata.iter() {
4     let value = &blob[cursor .. cursor + entry.x_size as usize];
5     let padded = (entry.x_size as usize + 7) & !7;
6     cursor += padded;
7     padded_total += padded;
8 }
9 assert_eq!(padded_total, blob_header.xf_used_data as usize);
```

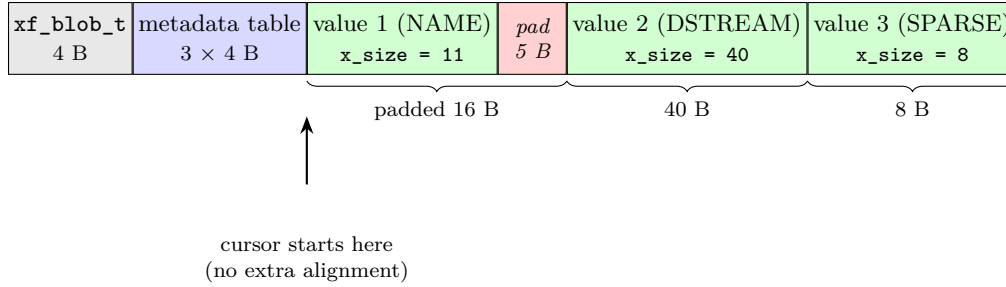


Figure 6.1: The xfield tail for the sparse inode at OID 23 (`xf_num_exts=3`, sizes 11/40/8). The cell widths are proportional to bytes. The value cursor starts immediately after the metadata table; each value advances by `round_up(x_size, 8)`. Total padded value-area length is $16+40+8 = 64$ B, matching `xf_used_data`.

6.2 Why The Reference Reads Otherwise

Apple's reference describes the xfield layout in terms that admit a "the value area is independently eight-byte aligned" reading. Under that reading, the cursor would jump to the next eight-byte boundary after the metadata table before reading the first value. The difference is small and frequently invisible: when `xf_num_exts * 4` is already a multiple of eight, both readings agree.

The converged code is uniform that no such re-alignment happens. The value cursor starts at the metadata-table end, full stop. Each value's padding to the next eight-byte multiple is performed *after* the value, not before the first.

The project initially modelled four candidate layouts in EX-13, scoring each against the proof fixture and selecting per-record. That probe successfully recovered the correct values but did not explain why. SR-015 reviewed the converged sources and showed that the four candidates were a probe artefact: a single rule explains every record the candidates fit. EX-16 re-ran the decode under the single rule and confirmed it.

6.3 Evidence From The Proof Fixture

Observation. On the proof fixture, 14 records carry xfield tails. Under the single cursor rule, every one of them satisfies `xf_used_data == sum(round_up(x_size, 8))`, and every one of them reconstructs the namespace and per-file logical size identically to the mounted POSIX oracle.

The records and their xfield shapes:

oid	family	num_exts	x_size set	xf_used_data	padded sum
2	inode	1	5	8	8
3	inode	1	12	16	16
16	inode	1	4	8	8
17	inode	1	4	8	8
17	dir_rec	1	8	8	8
17	dir_rec	1	8	8	8
18	inode	1	11	16	16
19	inode	2	15, 40	56	56
20	inode	2	10, 40	56	56
23	inode	3	11, 40, 8	64	64
24	inode	2	40, 10	56	56
26	inode	1	9	16	16
27	inode	2	17, 40	64	64
28	inode	2	17, 40	64	64

The sparse inode at OID 23 is the load-bearing row. It carries three xfields (**NAME=11**, **DSTREAM=40**, **SPARSE_BYTES=8**) with padded lengths summing to 64; the **NAME** value has a non-multiple-of-eight **x_size** (11), which forces the cursor to use padded advancement. Earlier candidate-scoring had needed per-record reasoning for this row; under the single rule it decodes cleanly.

No record has unused trailing bytes inside its xfield blob. The structural assertion holds without exception.

6.4 The Required Xfield Types

The xfield universe is large; an indexer needs four types and tolerates others.

INO_EXT_TYPE_NAME. The inode name. The name is also reachable through **DIR_REC** keys, and the indexer prefers **DIR_REC** names for namespace construction (they carry the directory placement). The inode-name xfield is a consistency check.

INO_EXT_TYPE_DSTREAM. A `j_dstream_t` embedded in the inode, carrying logical size, allocated size, default crypto id, and a default-extent reference.

INO_EXT_TYPE_SPARSE_BYTES. A count of bytes the sparse-allocation layer has not committed to disk. This is not a logical size; chapter 8 develops the distinction.

DREC_EXT_TYPE_SIBLING_ID. A directory record's sibling-group identifier for hard links. **SIBLING_MAP** closes the mapping from sibling ID to inode.

Other types (**INO_EXT_TYPE_FS_UUID**, **INO_EXT_TYPE_PURGEABLE_FLAGS**, and so on) appear in the wild but are not consumed by the indexer's emission rows. The cursor rule still reads past them correctly because the rule treats every value as opaque bytes of length **x_size** padded to a multiple of eight; only the values the parser actually inspects need a known type.

6.5 Field-Level Cross-Tool Equality

The xfield rule, the fixed-header decoders, and the fail-closed gates of the next chapter together specify the body decoder completely. The project verified the specification by running two independent decoders against the same raw container and comparing every field.

Observation. On the proof fixture, the Rust body decoder emits 53 records. An independent Python decoder, built from the same SR-015 rule and the same fail-closed boundary, emits 53 records. The two record lists key identically on (`node_paddr`, `entry_index`). Per-record field comparison finds zero divergent fields across `key`, `value`, every `xfields[*]`, `xfield_used_data`, `xfield_padded_total`, `xfield_unused_trailing_bytes`, `validation_notes`, `key_len`, `value_len`, `object_id`, `raw_type`, and `family`.

Cross-tool field equality is the strongest oracle the body decoder will see until the project has a same-run `go-apfs` body dump to add. The Python decoder and the Rust decoder are independent implementations of the same source-review rules; their agreement makes the rules themselves testable.

6.6 When To Use This Chapter

A reader implementing an APFS body decoder should:

1. read the fixed header for the record type from Apple’s reference,
2. apply the cursor rule above to the xfield tail,
3. validate against every fail-closed condition in chapter 7,
4. compare against an independent reference implementation, not against intuition.

Steps 3 and 4 are not optional. The next chapter is what they look like in practice.

Chapter 7

The Fail-Closed Boundary

A parser that emits rows for cases it does not understand is worse than a parser that emits nothing. The user can react to a missing row by looking elsewhere; a silently wrong row propagates into aggregates, into treemaps, and into actions the user takes on the basis of those aggregates. The cost of being silently wrong is unbounded. The cost of hard-stopping with a typed reason is bounded: the caller chooses a fallback, and no false output reaches the user.

Fail-closed is therefore not an aesthetic preference. It is the only discipline that bounds the cost of incomplete understanding.

7.1 Where The Boundary Sits

The boundary is concentrated in two single chokepoints:

Object headers. One function (`validate_object_block` in the Rust crate) validates every block before any reader trusts its body. It checks the Fletcher-64 checksum, the object type, the expected storage class, the encryption flag, the no-header flag, optional `o_oid == paddr` for physical objects, and the optional max-XID guard.

Record bodies. One function (`decode_fs_record`) validates every FS-tree leaf record against the rules below before any emission path trusts the decoded values.

There is no second path through either chokepoint. A block that fails header validation never reaches a body decoder; a record that fails body validation never reaches the namespace builder. The discipline is maintained by the topology of the calls, not by every caller remembering to validate.

A representative hard stop, from the body decoder:

```
1 if blob_header.xf_used_data as usize != padded_total {
2     return Err(ScanError::InvalidObject(format!(
3         "xf_used_data {} disagrees with padded value-area sum {}",
4         blob_header.xf_used_data, padded_total,
5     )));
6 }
```

The error carries the rule that failed, the observed value, and the expected value. The caller dispatches on the variant (`InvalidObject`), and the substring is preserved for diagnostic display.

7.2 The Per-Record Catalogue

Observation. Twenty-one per-record fail-closed cases are enforced by Rust unit tests on synthetic raw blocks. Each test asserts that the body decoder returns a typed `InvalidObject(reason)` with the rule’s substring.

7.2.1 Short bodies

1. Inode value shorter than the fixed `j_inode_val_t` struct.
2. Directory-record value shorter than the fixed `j_drec_val_t` struct.
3. Sibling-link value with `name_len` exceeding the bytes actually available.
4. Sibling-map value shorter than 8 bytes.

7.2.2 Malformed names

5. Variable-length name length longer than the bytes available.
6. Embedded NUL inside a required name (other than the null-terminator).
7. Non-UTF-8 bytes in a required name.
8. Directory-entry type outside the POSIX `DT_*` allowlist.

7.2.3 Malformed xfields

9. Duplicate xfield types inside one blob.
10. `xf_used_data` disagreeing with `sum(round_up(x_size, 8))`.
11. Xfield value cursor running past the blob.
12. Xfield metadata table running past the blob.
13. Xfield blob shorter than the 4-byte `xf_blob_t` header.
14. Required xfield with the wrong `x_size` (`DSTREAM` of length other than 40).
15. Required xfield with the wrong `x_size` (`SIBLING_ID` of length other than 8).

7.2.4 Malformed xattrs

16. Xattr value shorter than the fixed header.
17. Xattr sets both `EMBEDDED` and `STREAM` flags.
18. Xattr sets neither flag.
19. Xattr sets unknown flag bits.
20. Xattr body length disagrees with `xdata_len`.
21. Xattr stream body shorter than `j_xattr_dstream_t`.

7.3 The Cross-Record Cases

Three SR-016 cases depend on consistency across more than one record and therefore cannot be enforced by synthetic single-record tests. They are deferred to fixture-level probes; the open questions chapter tracks their status.

Drec type vs inode mode. A directory entry's `DT_*` type and the referenced inode's `S_IFMT` mode must agree. A regular-file inode pointed at by a `DT_DIR` entry is malformed at the volume level even though both records validate in isolation.

Sibling-map closure. A directory entry that carries `DREC_EXT_TYPE_SIBLING_ID` requires a `SIBLING_MAP` record that maps that sibling ID to the entry's child file ID. A missing closure produces a hard-link group with a dangling edge.

Compressed-size precedence conflict. An inode that sets `INODE_HAS_UNCOMPRESSED_SIZE` but does not carry a `com.apple.decmpfs` xattr is internally inconsistent at the cross-record level. Chapter 8 develops the precedence rule that prevents the conflict from producing wrong rows.

These cases will close as the project lands EX-22 and the future cross-record consistency probe.

7.4 Why The Boundary Is Tighter Than It Looks

The fail-closed catalogue may seem heavy for a parser that, in ordinary operation, encounters none of these cases. The point of the catalogue is what happens in the not-ordinary case.

Three concrete situations exercise the boundary in practice:

Format drift. APFS reserves bits and types over time. A container written by a newer macOS may carry an xattr flag, an xfield type, or an incompatible-feature bit the parser does not recognise. The unknown-bit hard stops mean a newer container produces a typed refusal rather than a quietly wrong row.

Genuinely malformed images. Recovery scenarios, partial image dumps, or test corpora may include containers with structurally inconsistent records. The catalogue refuses them with a precise reason, which the caller can present as a diagnostic.

Parser bugs. If a future change to the body decoder silently re-introduces an off-by-one in the xfield cursor, the `xf_used_data` structural assertion catches it on the first record with a non-multiple-of-eight `x_size`. The catalogue is also a regression suite.

The catalogue is therefore proportional to the cost it prevents, not to the frequency with which it is exercised.

7.5 The Object-Header Side

The header-validation chokepoint is smaller but no less load-bearing. A block that does not validate is never decoded; a block that validates under the wrong storage class is also rejected, because `validate_object_block` takes the expected storage as a parameter.

The storage classes the parser uses:

Physical. The object's `o_oid` equals its physical block address. Container superblocks, checkpoint maps, OMAP nodes, and free-queue nodes are physical.

Virtual. The object lives in a virtual address space resolved by an OMAP. `o_oid` is independent of the block's `paddr`; the validator checks `o_oid == omap_lookup.oid` instead. Volume superblocks (looked up through the container OMAP), FS-tree roots and internal nodes (looked up through the volume OMAP), and dstream extent nodes are virtual.

Ephemeral. In-memory between transactions; reached through the checkpoint map. The parser uses the checkpoint map's mapping, never an ephemeral storage flag.

Mixing classes is a structural mistake. Chapter 3 records the empirical correction that checkpoint-map *blocks* are themselves physical, not ephemeral, despite naming objects that are ephemeral; chapter 5 records that FS-tree roots are virtual despite living at a known `paddr` that the volume OMAP returns. The parser keeps both rules isolated to their call sites.

7.6 The Boundary And The Fallback

The fail-closed boundary is the upstream half of the support contract; the fallback path (chapter 11) is the downstream half. When the boundary refuses a source, the caller is not stuck: it dispatches to the POSIX-traversal fallback, which produces the same shape of output through supported APIs. The user sees an answer; the wrong answer is not produced.

This is the design pattern the rest of the manual presupposes: known behaviour is enforced as a hard rule, unknown behaviour is escalated to a path that asks the operating system rather than guessing on its behalf.

Part IV

Names and Sizes

Chapter 8

Sizes and Their Precedences

APFS records the size of a file in several places, and the places do not always agree. A parser that picks the wrong source on the wrong file silently produces wrong rows and wrong directory totals. This chapter fixes two rules — one for logical size, one for allocated size — and shows the cases for which each is the correct rule.

The two metrics are independent. They have separate sources, separate oracles, and separate fail-closed boundaries. The chapter develops them in that order: logical first (the primary anchor; covers every case), allocated second (covers five of seven case classes; sparse and decmpfs fail closed).

8.1 Logical Size

The primary size metric is *logical size*: the user-visible byte length, the number POSIX `st_size` reports, the number the shell shows in `ls -l`. It is not the bytes on disk; it is not the bytes the file would occupy after decompression on a different filesystem; it is not the bytes uniquely owned by this inode. Each of those is a separate metric, and at most one is treated here (allocated size, in the next section).

The choice of logical size as the primary anchor is deliberate. It has a stable oracle (`st_size`), applies uniformly to ordinary, sparse, cloned, hard-linked, symlink, and compressed files, and is the size every user-facing tool already agrees on.

8.1.1 The Precedence Rule

Observation. Five steps, applied per inode, produce `st_size` for every case the validated corpus covers:

1. If the inode's `internal_flags` carry `INODE_HAS_UNCOMPRESSED_SIZE`: pick `inode.uncompressed_size`.
2. Else if the inode carries a `j_dstream_t` (via `INO_EXT_TYPE_DSTREAM` or a paired `j_dstream_id_val_t` record): pick `j_dstream_t.size`.
3. Else if a `com.apple.decmpfs` xattr is present and step 1 did not fire: pick the decmpfs header's `uncompressed_size` field.
4. For symlinks (`S_IFLNK`): pick the byte length of the `com.apple.fs.symlink` xattr payload.
5. Otherwise: zero.

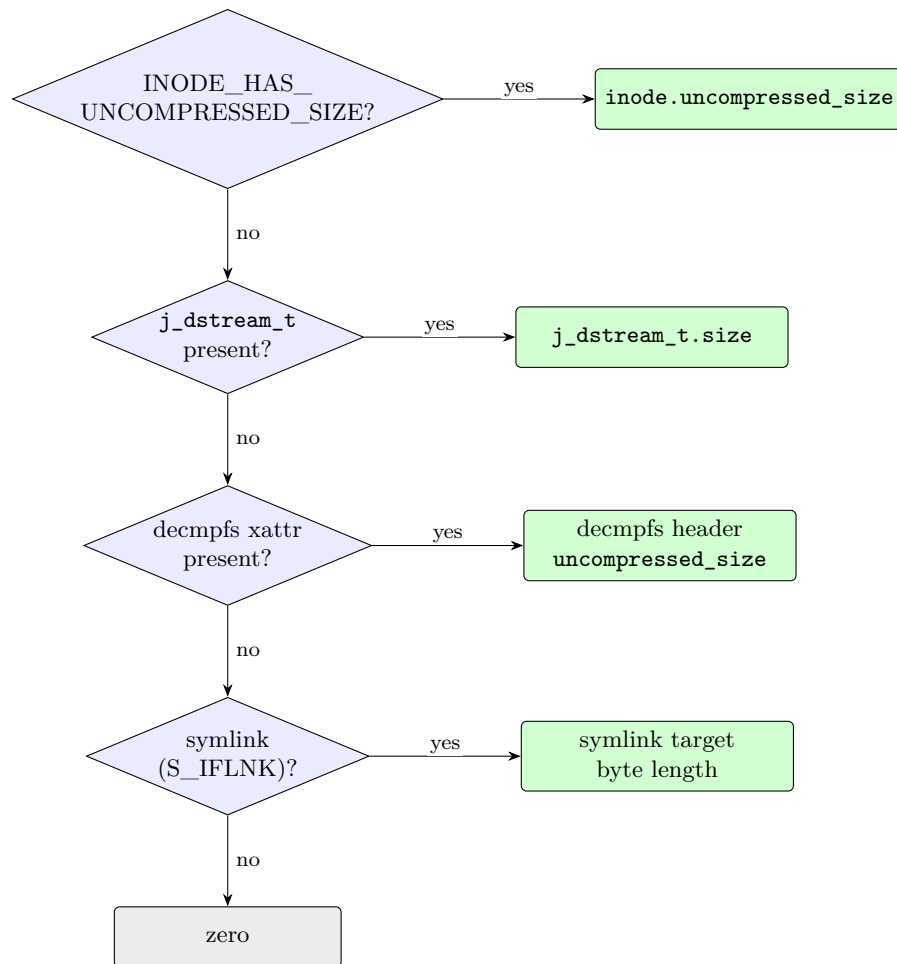


Figure 8.1: Per-inode logical-size precedence. The order is non-negotiable; the compressed case is the load-bearing reason step 1 must beat step 3.

The order matters. Steps 1 and 3 both reach into compression metadata, but they reach for different fields and have different trust. The next subsection shows why.

In pseudocode the same rule reads as one short function:

```

1 def logical_size(inode, xattrs):
2     if inode.internal_flags & INODE_HAS_UNCOMPRESSED_SIZE:
3         return inode.uncompressed_size
4     if inode.dstream is not None:
5         return inode.dstream.size
6     decmpfs = xattrs.get("com.apple.decmpfs")
7     if decmpfs is not None:
8         return decmpfs.header.uncompressed_size
9     if S_ISLNK(inode.mode):
10        return len(xattrs["com.apple.fs.symlink"].payload)
11    return 0
  
```

8.1.2 The Load-Bearing Case: Compression

Observation. On the EX-19 fixture the compressed file (`compressed.txt`, created via `ditto -hfsCompression`) presents:

- `INODE_HAS_UNCOMPRESSED_SIZE` set, `inode.uncompressed_size = 53248`.
- No `j_dstream_t`. APFS stores small compressed data inline in the `decmpfs xattr` and does not allocate a separate `dstream` for it.
- `com.apple.decmpfs xattr` present, with header `uncompressed_size = 437` and `compression_type = 0`.
- Mounted POSIX `st_size = 53248`.

The `decmpfs` header's `uncompressed_size` is a placeholder for this layout. Step 1 must beat step 3, or the parser emits 437 instead of 53248. The fixture is small but the principle is general: when APFS sets the inode flag, the inode flag is the source of truth; the `decmpfs` header is metadata the running system maintains for its own decompression path and is not reliably the file's size.

8.1.3 The Hint That Is Not A Size

Observation. `INO_EXT_TYPE_SPARSE_BYTES` is an *allocation hint*, not a logical size. It records the bytes the sparse-allocation layer has not committed to disk.

On the proof fixture's sparse file:

- `j_dstream_t.size = 1052897` (which equals `st_size`).
- `INO_EXT_TYPE_SPARSE_BYTES = 1032192`.
- `st_size = 1052897`.

The sparse bytes number describes the difference between the file's logical extent and its allocated bytes. It is useful for the allocated-size metric (next section) but is not the answer to "how large is this file"; that number is the `dstream` size, which the precedence rule correctly picks at step 2.

A reader that treats `SPARSE_BYTES` as anything else is adding sparse holes to the logical size of every sparse file — a sizeable error on real-world data.

8.1.4 Per-Inode Results

Observation. On the EX-19 same-run fixture covering ordinary, sparse, clone, hard-linked, symlink, and compressed cases, the precedence rule reproduces `st_size` for every unique inode:

path	inode	step picked	value	st_size
ordinary.txt	16	<code>j_dstream_size</code>	29	29
sparse.bin	19	<code>j_dstream_size</code>	1052897	1052897
clone.txt	20	<code>j_dstream_size</code>	29	29
link.txt	23	<code>symlink_target_len</code>	12	12
compressed.txt	24	<code>inode_uncompressed_size</code>	53248	53248

Zero mismatches. The compressed case picks step 1; everything else picks step 2 except the symlink (step 4). The hard link does not appear as a separate row because the rule is per inode and `hard.txt` shares inode 16 with `ordinary.txt`.

8.2 Allocated Size

Per inode, the number of bytes APFS has allocated on disk for the file’s content stream.

Observation. For ordinary, cloned, hard-linked, and symlink cases the parser emits `j_dstream_t.allocated_size` (or zero for symlinks) and the result matches macOS’s `st_blocks * 512` oracle. Two further cases used to fail closed — sparse files and decmpfs-compressed files — and were closed by EX-26 against the same `st_blocks * 512` oracle. For sparse, the allocated bytes are `allocated_size - sparse_bytes`. For decmpfs, the allocated bytes are the sum of any stream-backed xattrs’ `allocated_size` values (the primary dstream is usually absent, and the compressed bytes live in either `com.apple.decmpfs` or `com.apple.ResourceFork`; an embedded xattr contributes zero because the bytes live inline in the xattr payload).

8.2.1 The Precedence Rule

Observation. Six branches, applied per inode:

1. Regular file with a `j_dstream_t` xfield and no `INO_EXT_TYPE_SPARSE_BYTES` xfield and no `com.apple.decmpfs` xattr: emit `Some(j_dstream_t.allocated_size)`.
2. Regular file with a `j_dstream_t` xfield and a `SPARSE_BYTES` xfield (no decmpfs): emit `Some(allocated_size - sparse_bytes)` (EX-26 Hypothesis A, see §8.2.3).
3. Regular file with a `com.apple.decmpfs` xattr: emit `Some(primary + decmpfs.stream_dstream.allocated + ResourceFork.stream_dstream.allocated)` (each term defaults to zero when absent or embedded; EX-26 Hypothesis F, see §8.2.4).
4. Symlinks: emit `Some(0)`.
5. Directories: emit `Some(0)`.
6. Anything else: emit `None`.

`None` is not a synonym for zero. A row with `allocated_size = None` is recorded in the run’s `not_claimed` register, the metric toggle in the visualisation greys out for that row, and any directory aggregate that contains a `None`-allocated inode is itself emitted as `None` — a partial aggregate is forbidden because a consumer might otherwise treat it as authoritative.

8.2.2 Why `allocated_size` Is The Only Candidate

Six independent open-source readers were surveyed (`linux-apfs-rw`, `apfsprogs/apfsck`, the Sleuth Kit, `libfsapfs`, `dissect.apfs`, `apfs-fuse`, `go-apfs`). All six decode `j_dstream_t.allocated_size`; only TSK and `linux-apfs-rw` surface it to callers, and `linux-apfs-rw` publishes it directly as `inode->i_blocks << 9`. No reader surfaces the alternative candidate — the sum of `j_file_extent_val_t.len` over the inode’s extents — as a public allocated-bytes number. The extent-reference tree (`OBJECT_TYPE_BLOCKREFTREE`) is opened by some readers but is the per-volume reference-counting authority for snapshots and clones, not a per-file allocated-bytes table.

A consistency conflict surfaces between two halves of the same project: `linux-apfs-rw` (the kernel module) writes `allocated_size = round_up(ds_size, blocksize)`; `apfsck` (the same project’s filesystem checker) enforces `allocated_size == sum(j_file_extent_val_t.len)`. The two agree only when every extent ends exactly at `ds_size` and the file has no truncate residue. The kernel

can therefore write volumes that its companion checker flags as corrupt. The right oracle is the external public number (`st_blocks * 512`), not internal field consistency.

8.2.3 The Sparse-File Lift (EX-26 Hypothesis A)

EX-22 left this branch fail-closed because the fixture carried a single sparse shape and the observed identity `allocated_size - sparse_bytes = st_blocks * 512` was a hypothesis worth a corpus-level probe before promoting to rule. EX-26 ran that probe on four sparse shapes and the identity held exactly across all of them.

Observation. On the EX-22 fixture the sparse file carries:

- `j_dstream_t.size = 1052897` (equals `st_size`).
- `j_dstream_t.allocated_size = 1056768` (`= round_up(size, 4096)`).
- `INO_EXT_TYPE_SPARSE_BYTES = 1032192` (the unallocated bytes inside the logical extent).
- `st_blocks = 48`, so `st_blocks * 512 = 24576`.

`allocated_size` overstates the actual on-disk allocation by exactly $1,056,768 - 24,576 = 1,032,192$ bytes — the sparse-bytes hint. APFS on macOS follows the `linux-apfs-rw` kernel rule rather than the `apfsck` sum-of-extents rule. The emission rule must therefore split the dstream case by presence of the `SPARSE_BYTES` xfield.

EX-26 corpus result. The identity `allocated_size - sparse_bytes = st_blocks * 512` holds exactly across four sparse shapes:

shape	allocated_size	sparse_bytes	difference	st_blocks * 512
1 MiB HEAD/TAIL hole	1,056,768	1,032,192	24,576	24,576
10 MiB hole	10,485,760	10,452,992	32,768	32,768
50 MiB hole	52,428,800	52,396,032	32,768	32,768
4 KiB-data-per-64 KiB chunked	2,097,152	1,572,864	524,288	524,288

The branch now emits `Some(allocated_size.saturating_sub(sparse_bytes))`, with saturation guarding the (not observed in practice but algebraically possible) case where the kernel's residue exceeds the dstream- reported allocation.

8.2.4 The decmpfs Lift (EX-26 Hypothesis F)

EX-22 left `decmpfs` fail-closed because none of the open-source readers agreed on the macOS-oracle number and the EX-22 fixture carried only one `decmpfs` case (the small-payload `xattr-stream-stored` shape). EX-26 added a second `decmpfs` variant (a 256 KiB compressible payload that ditto routes into the `resource-fork-` stored shape) and observed that both shapes are captured by a single rule: the allocated bytes are the sum of any stream-backed `xattr` `allocated_size` values plus the (typically absent) primary dstream `allocated_size`.

Observation. On the EX-26 fixture the two `decmpfs` files carry:

- `compressed.txt` (52 KiB compressible JSON): no primary dstream; `com.apple.decmpfs` is *stream-backed* (`flags = 0x01`) with `stream_dstream.allocated_size = 4096`; `com.apple.ResourceFork` is stream-backed with `stream_dstream.allocated_size = 0` (empty fork). Sum: 4096. Oracle `st_blocks * 512 = 4096`.

- `compressed-big.bin` (256 KiB compressible binary): no primary dstream; `com.apple.decmpfs` is *embedded* (flags = 0x02) with payload `fpmc` magic, `compression_type` = 8 (lzvn resource-fork-stored), `uncompressed_size` = 262144; `com.apple.ResourceFork` is stream-backed with `stream_dstream.allocated_size` = 4096. Sum: 4096. Oracle `st_blocks * 512` = 4096.

The on-disk reality is that the `compression_type` byte in the `decmpfs` header tells you *where* the compressed bytes live (xattr inline vs. resource fork), but the bytes themselves are always counted by summing the `stream_dstream.allocated_size` fields of every xattr that is stream-backed. Embedded xattrs contribute zero because their bytes live inline in the xattr payload, which is part of the inode body’s storage and does not allocate separate extents.

The disagreement between `linux-apfs-rw` (publishes uncompressed size / 512) and `apfs-fuse` (publishes zero) that EX-22 documented is now resolved against the macOS oracle: neither was right; the sum-of-stream-dstreams formula is the public on-disk rule that reproduces `st_blocks * 512` exactly.

A residual case the EX-26 fixture did not exercise is the *incompressible-payload* shape, where ditto leaves `UF_COMPRESSED` off and emits no `com.apple.decmpfs` xattr at all. Such files fall through to branch 1 (regular file with a primary dstream) and emit `Some(allocated_size)` under the EX-22 baseline; on the EX-26 fixture the `compressed-random.bin` case did exactly this.

8.2.5 Per-Inode Results

Observation. On the EX-26 fixture, all ten rows match the `st_blocks * 512` oracle exactly:

path	branch	rule	picked
ordinary.txt	1	<code>allocated_size</code>	4096
clone.txt	1	<code>allocated_size</code>	4096
compressed-random.bin	1	<code>allocated_size</code>	131,072
sparse.bin	2	<code>allocated_size - sparse_bytes</code>	24,576
sparse-medium.bin	2	<code>allocated_size - sparse_bytes</code>	32,768
sparse-large.bin	2	<code>allocated_size - sparse_bytes</code>	32,768
sparse-chunked.bin	2	<code>allocated_size - sparse_bytes</code>	524,288
compressed.txt	3	<code>decmpfs.stream_dstream.allocated</code> (xattr-stream-stored)	4096
compressed-big.bin	3	<code>ResourceFork.stream_dstream.allocated</code> (resource-fork-stored)	4096
link.txt	4	<code>zero</code> (symlink)	0

Zero mismatches; every row picks a value the rule emits, against the macOS oracle. The hard-linked file `hard.txt` does not appear as a separate row because it shares its inode with `ordinary.txt` and contributes through the same per-inode rule.

8.3 Directory Aggregates

A directory’s total is the *unique-inode logical sum within the aggregate root*.

A regular file’s logical size contributes to a directory total once, regardless of how many hard-linked paths within that directory tree reference the same inode. A symlink contributes its target byte length to its own row but contributes nothing to any directory total (the target may live elsewhere or not exist at all, and even when it exists its size is already counted at the target’s row).

Why this policy. The naive policy — sum the logical size of every path under a directory — catastrophically over-counts on real directories with hard-linked content: mail spools, package caches, Git object stores, snapshot archives. The user looking at a 12 GB directory deserves to know that 11 GB of it is a single 11 GB file that also appears elsewhere, not that the directory legitimately holds 12 GB of unique content. WizTree-on-NTFS rarely encounters hard-link density that exposes this; on APFS the case is common.

Consequence. Sibling directory totals are not strictly additive into their parent when the same file identity appears in more than one subtree. This is correct, not buggy, but the user interface must label aggregate semantics so the user understands what the number means.

The aggregate rule is implemented in one place (`aggregate.py`, ported to the Rust crate’s `namespace` module) using `setdefault(file_id, logical_size)` keyed by inode — the policy is enforced by construction rather than by careful summing.

Allocated-size aggregates use the same unique-inode rule, with the extra step that any contributing inode whose `allocated_size` is `None` (sparse or decmpfs) propagates `None` up to the directory total.

8.4 What This Chapter Does Not Promise

Three metrics remain outside the scope this chapter defends. Each has its own oracle and its own corpus of edge cases; chapter 13 records them as open questions for future work.

Exclusive size. Bytes owned only by this inode, after accounting for clones and snapshots. Requires the extent-reference tree to be opened and joined against per-inode extents.

Shared size. Bytes shared with at least one other inode or snapshot.

Snapshot-retained bytes. Bytes that exist on disk solely because a snapshot still references them.

A future version of this manual will have its own chapter on each. Until those chapters exist, the parser does not emit those columns. Logical size is what it knows; allocated size is what it knows for five-of-six case classes; sparse and decmpfs allocated sizes are what it refuses to claim.

Chapter 9

Names Across Volume Modes

A row in the namespace is identified by a path, and a path is a sequence of names. The names come from directory record keys; the keys are stored as UTF-8 byte strings. The chapter develops one finding and one deliberate non-promise.

9.1 The Finding

Observation. Stored names pass through verbatim.

The Rust body decoder emits the bytes the directory key stores. The parser does not normalise, case-fold, or interpret. POSIX traversal of the same volume returns the same bytes. The two agree byte for byte.

The finding is stated in the simplest form on purpose. Anything more elaborate — "Rust emits the canonical form" or "Rust converts to NFC" — would be wrong on some volume and would silently rename files in the output. Emit what the volume stored; the user can compare what they see to what `ls` shows.

9.2 Two Volume Modes Exist

APFS volumes come in two case-and-normalisation modes that the indexer must handle.

APFS (case-insensitive). `case_insensitive = true, normalization_insensitive = false.`

The volume's incompatible-feature mask sets the case-insensitive bit only. The volume is also normalisation-insensitive for lookup, but APFS reserves the explicit incompat bit for the case-sensitive variant. The asymmetry is a quirk of the on-disk format, not a statement about behaviour.

Case-sensitive APFS. `case_insensitive = false, normalization_insensitive = true.`

Both modes appear on real systems, and both have to be supported.

9.3 What The Modes Do Affect

The modes affect *lookup*: when the kernel resolves a path, it consults the volume's case and normalisation behaviour to decide whether two names refer to the same file. They do not affect *row enumeration*: enumeration walks directory keys and returns their bytes.

The distinction is the chapter’s central claim. The parser can emit correct rows on both modes without performing the lookup-equivalence test, because the lookup behaviour is the running system’s responsibility, not the parser’s.

9.3.1 The Lookup Oracle

Observation. Across the two volume modes, APFS lookup behaves as follows. On the EX-20 same-run fixture using `os.O_CREAT | os.O_EXCL`:

operation	CI volume	CS volume
create <code>plain.txt</code>	accept	accept
create <code>CaseName.txt</code>	accept	accept
<code>casename.txt</code> after <code>CaseName.txt</code>	EEXIST	accept
create NFC <code>café.txt</code>	accept	accept
NFD <code>café.txt</code> after NFC sibling	EEXIST	EEXIST

The case-insensitive volume rejects the lowercase form because lookup considers the two names equivalent. Both volumes reject the decomposed Unicode form after the precomposed sibling because both treat NFC and NFD as the same name for lookup. This is what APFS does; no APFS-specific knowledge in the indexer is needed to produce the result, because the kernel did the work.

9.3.2 Rust Vs POSIX Equality

Observation. On both volumes the Rust-emitted path bytes match POSIX-traversal path bytes exactly. The comparison is byte for byte: `path.encode("utf-8").hex()` produces the same string on both sides for every entry. On the CI volume the corpus has four entries; on the CS volume it has five (the case-distinct sibling also appears); the equality holds for every one of them.

The cleanest way to see this: both sides walk the directory key storage. POSIX walks through the kernel; Rust walks through the FS tree directly. Neither side applies normalisation, and the bytes are the same bytes either way.

9.4 The Deliberate Non-Promise

The parser does not implement APFS lookup-by-name.

The APFS lookup primitive is the directory hash: a CRC32C over the name’s UTF-32LE codepoints, with NFD normalisation applied to the input first, and on case-insensitive volumes Unicode case-fold applied before normalisation. Two names with the same hash collide within a directory; APFS resolves the collision by storing both keys and walking them.

A *find this exact path* feature has to reproduce that hash to locate the directory entry. The project has not yet validated a hash implementation against a same-run hash oracle, and until it does, the parser does not pretend to find files by name.

The asymmetry is deliberate. A row enumeration that is correct — every visible path appears once, with the right bytes — is useful immediately: it powers a treemap, sort-by-size, drill-down, and context-menu operations on rows the user has selected. A lookup-by-name that is silently wrong returns the user’s query against the wrong file. The cost of being wrong about a lookup is unbounded; the cost of refusing to do the lookup is "the user has to navigate to the file rather than type its name." The asymmetry of cost dictates the asymmetry of scope.

9.5 Names And Aggregates

The name preservation rule has one further consequence. Directory aggregates (chapter 8) key on inode identity, not on path. Two paths that hard-link to the same inode contribute that inode's logical size once to a directory total, regardless of whether their names are casewise or normalisationally different.

The hard-link unification reads `SIBLING_LINK` and `SIBLING_MAP` records to find every name that points at a given inode. The names are emitted verbatim in each row; the inode is recorded as the row's `file_id`; the aggregate keys on the `file_id`. Names and sizes are decoupled from aggregate identity, which is the right shape for a filesystem that allows the same content to live under many names.

9.6 What Future Work Will Need

A future lookup-by-name feature will need:

1. A same-run hash oracle: a probe that takes a constructed name, runs APFS's own hash through a controlled API (the kernel's `O_EXCL` accept/reject is the cheapest indicator), and produces a positive case and a negative case for the hash equivalence class.
2. NFD normalisation tables and Unicode case-fold tables that match the macOS version the volume was written on, accounting for tables changing across Unicode revisions.
3. Collision handling: directories may store multiple keys with the same hash, and the lookup must walk all of them.
4. Volume-mode dispatch: case-insensitive volumes apply case-fold before normalisation; case-sensitive volumes do not.

That feature has its own oracle and its own probe sequence and is out of scope. The non-promise here is what keeps the namespace claim defensible.

Part V

Boundaries

Chapter 10

Identity, Reuse, and What a Cache Key Cannot Be

The most attractive idea in any fast-indexer project is that a second scan should be faster than the first. The natural mechanism is a cache of parsed subtree summaries: walk the FS tree once, summarise each node, and on the next run reuse the summary if the node has not changed. The chapter is about why the obvious cache key for that mechanism is dangerous, and what the right shape of a safe key looks like.

10.1 The Tempting Mistake

The most obvious cache key is the OID: it identifies an object across versions, it is stable across copy-on-write rewrites, and it is small. A summary cache keyed on `(volume_uuid, oid)` would deduplicate unchanged subtrees across runs.

It would also return stale summaries, silently, the first time the volume mutates a subtree the cache already knows.

Observation. The FS-tree root's logical OID stays stable across mutations; everything else about the on-disk object changes. EX-06 mutated the proof fixture, recorded the FS-tree root identity fields before and after, and observed:

field	across mutation
logical OID	stable
physical address (paddr)	changed
object XID	changed
object checksum	changed
block hash	changed

A cache keyed on OID would return the pre-mutation summary for the post-mutation tree. The parser would emit rows derived from the old state and aggregate them into the new state's directory totals. The user would see a treemap that does not match the disk.

The OID is a *virtual address*. It names *some* version of the object; it does not name a particular content. A cache that wants content identity must use a content identifier.

10.2 The Shape Of A Safe Cache Key

A summary cache that is safe under mutation must distinguish entries that share an OID but differ in content. The minimum sufficient key is a tuple whose members come from several layers:

Source identity. A volume UUID, a container UUID, and an image fingerprint (for detached sources). Distinguishes one APFS volume from another, and distinguishes the same volume's different image dumps.

Parser version. A monotonic identifier for the body decoder and the namespace builder. A parser change invalidates every summary, because a summary's interpretation depends on the parser that produced it.

Summary schema version. A monotonic identifier for the summary structure itself. A schema change is independent of a parser change.

OMAP context. Whether the OID was resolved through the container OMAP or the volume OMAP. A collision between the two name spaces is rare but representable.

OID. The virtual identifier.

Object XID. The version of the object the summary was built for.

Physical address. Useful as a cross-check; an OMAP that resolves (oid, xid) to a different paddr than the cache expects invalidates the entry.

Checksum or content hash. The strongest individual signal. A block whose checksum has changed must not reuse a summary keyed on the old checksum.

Expected type and subtype. Defensive. An OID that has been re-allocated to a different object class is a new object.

The key is therefore not a small integer. It is a record of nine fields, eight of which the parser already needs to compute for any read, and the ninth (parser version) is constant per build.

10.3 What EX-07 Proved And What It Did Not

The project's first probe of the cache idea (EX-07) restricted itself to a narrow question: under exact node-identity matches on every field of the key above, does subtree reuse ever produce a wrong summary?

Observation. EX-07 found zero false reuse for exact node-identity matches in the detached lab corpus.

That is the strongest positive result a subtree-reuse cache can ask for at this stage. It means the mathematical foundation is sound: if two parses see the same node identity, they will get the same summary, and reusing one for the other is safe.

It is not yet a production cache theorem. The probe ran on one corpus, on detached images, with mutations limited to a small set. Three things still have to land before a cache is safe to ship:

1. Re-run EX-07's mutation grid against the native Rust full parse (now possible, since the parser and the parallel fallback of chapter 11 are both production code) to confirm the finding holds when the parser is this project's own production code rather than a third-party reference.

2. Build a cache storage design with crash-safe writes; a cache that can corrupt during a kill produces stale-and-wrong rather than absent-and-correct, which is the failure mode the fail-closed discipline is supposed to prevent.
3. Specify an invalidation policy that names every condition under which a summary becomes stale: parser version mismatch, missing volume continuity, source identity mismatch, unsupported feature drift, checkpoint rollback, snapshot switch, unknown record-body type, or any new metric the cache did not record (the allocated-size column is the most recent example of a column a pre-existing cache would not have recorded).

Until those gates pass, the safe default is full reparse, validated against the namespace, logical-size, and allocated-size oracles. A second scan of the same volume produces the same answer as the first; the parallel walker has made it produce that answer faster, but it does not yet produce it incrementally.

10.4 The Identity-Tracking Discipline

A small detail of EX-06 deserves promotion. The probe did not assume which identity fields would change; it recorded all of them, every time. That habit — "record every plausibly-load-bearing identity field at every relevant boundary, before deciding which ones matter" — is what made the cache discussion concrete instead of speculative.

The same habit appears in EX-12's OMAP lookup contract: the probe records every resolved (oid, paddr, xid) sample and replays header validation at each, rather than checking only the resolutions that the parser happens to read. The cost of recording is small; the cost of discovering, later, that a key field was never captured is large.

The discipline generalises beyond caching. A reverse-engineering project produces evidence whose value depends on what it recorded at the time. The right rule is: when the question is "which fields matter," the experiment records all of them.

10.5 What This Chapter Forecloses

The chapter forecloses two specific design moves that other indexers sometimes make.

No bare-OID cache. The parser will never key reuse on OID alone, even with parser-version qualification. The mutation behaviour above means a bare-OID cache is unsafe in principle, not just unsafe under unusual conditions.

No partial summary reuse across mismatched scan XIDs. A summary computed at scan XID *a* is not reused at scan XID *b*. The whole point of pinning the scan XID is that object identity includes the XID; relaxing that turns the cache into a stale-state producer. The cache may compare keys across runs only when the keys agree on every field, scan XID included.

The chapter does not foreclose the cache idea itself. Subtree reuse is a real opportunity — the EX-07 result is encouraging — but the cost of getting it wrong is high enough that the project will land it only after the gates above are met. Until then, every scan fully reparses, and the manual records why.

Chapter 11

Support, Readability, and the Fallback

The chapter develops a vocabulary the rest of the manual has used implicitly: what does it mean to "support" a source. The vocabulary matters because every product decision sits on top of it. A source that is supported is one the product makes a correctness claim on; a source that is merely readable is not. The two are easy to confuse.

11.1 Four Properties, In Order

Observation. Four properties characterise a source's relationship to the parser. The order matters; each builds on the last.

Readable. Raw bytes can be opened and sampled. A device file exists and `read()` returns data.

Parsable. Checkpoint and root discovery succeed; raw traversal runs without aborting.

Validatable. The parser's output can be compared to a named oracle for the requested mode. A namespace can be diffed against a POSIX traversal; a logical size against `st_size`.

Supported. The source is in the product allowlist for the requested mode. Validation has been performed; results were correct; the source class belongs in production.

The first three are technical. The fourth is editorial: it is the project's decision about which sources are stable enough to make a correctness claim on. A source can be readable and parsable and validatable in one moment and not be supported because the moment is not durable.

11.2 Readable Is Not Supported

The common confusion is between readability and support. A mounted live volume is readable. It may, on a still moment, be parsable. It may, against a momentary POSIX snapshot, even be validatable. It is not supported, because the validation cannot be stable: the next moment moves the visible checkpoint, the next POSIX snapshot disagrees with the parser snapshot, and the user has no way to tell which row is from which moment.

The right rule: support a source class only when the validation recipe is reproducible. If the validation works once and then is not reproducible because the source state has moved, the source class is not supported even though one validation passed.

11.3 The Current Allowlist

The raw-mode allowlist is intentionally short.

- Detached, unencrypted, image-backed APFS containers. The `.dmg` layer fixes the visible state at detach time.
- Caller-supplied raw APFS container devices under `/dev/disk*` or `/dev/rdisk*` where the caller has committed that the checkpoint will not advance.
- Explicitly stable APFS sources — a tightly controlled lab volume where the operator pins the state.

What is not on the list: live mounted startup disks, FileVault volumes the user is using, multi-device or Fusion containers, sealed system volumes, snapshots opened through the operating system’s snapshot framework, and any container whose `omap_phys.flags` indicate an encryption operation in flight.

The allowlist will grow only after a paired oracle and a new probe clear the corresponding entry in the support matrix.

11.4 EX-28 Live-Volume Raw Mode (Hypothesis C: Blocked by the Kernel)

A live mounted volume’s `/dev/diskNsM` device node is not on the allowlist above. It cannot be: a live mounted volume is exactly the case the source-gate refuses, because the volume metadata advances during the scan window and SR-014 checkpoint selection has no way to pin the visible state.

EX-28 asks the narrower question: *if a user with root privileges hands the existing raw parser a `/dev/diskNsM` of a live volume, does the parser produce a self-consistent shape, and does that shape agree with what the fallback walker emits from the same mount point in the same scan window?*

The harness lands as a Rust integration test in `crates/apfs-fastindex/tests/ex28_live_parity.rs`, gated on the environment variables `APFS_FASTINDEX_EX28_LIVE_DEVICE` and `APFS_FASTINDEX_EX28_MOUNT_POINT`. Without those, the test is a clean no-op and `cargo test -release` skips it. With them set and the test run as root, the harness:

1. Tries three successive raw scans of the device. Each must publish a `selected_checkpoint` (no SR-014 fail-closure under live concurrent writes), with pairwise symmetric difference of their `NamespaceEntry` sets ≤ 200 paths.
2. Runs one fallback-walker scan of the mount point and compares it against the raw scan; symmetric difference must be ≤ 1000 paths to absorb the raw-can-read-paths-the-fallback-cannot asymmetry (TCC, sealed system volume reads).

The comparator behind both checks is `apfs_fastindex::parity::compare_namespace_shapes`, which returns a structured `ShapeDiff` with `only_in_left`, `only_in_right`, and per-field `mismatches`. It is exposed as a public crate surface so any future tooling can use it.

Observation (2026-05-20). The first privileged run on the project owner’s Apple silicon host returned the harshest of the planned fallbacks: macOS refused to even open `/dev/disk3s1` for raw reads, returning `EPERM` (errno 1, “Operation not permitted”) on the first `read(2)` of

the device’s block zero, despite the process running as root. The harness classifies this as `LiveScanOutcome::BlockedByKernel` and records the `live_raw_blocked_by_kernel` verdict. The kernel’s storage-system security policy refuses raw block access to the live boot data partition independently of POSIX file permissions; SIP and the sealed-system-volume seal both contribute. Neither the successive-scan stability nor the raw-vs-fallback parity assertions could be evaluated because the raw backend never opened.

Product implication. The "Scan as administrator" menu item the original EX-28 plan anticipated does not graduate to raw mode on a stock Apple silicon macOS host. The privileged-subprocess shape still serves a purpose — unlocking TCC-restricted user-data paths the fallback walker hits `EACCES` on — but it stays on the fallback walker even under root. The raw fast path is unchanged in its original scope: detached `.dmg` images, caller-pinned raw devices on a quiesced volume. EX-28’s contribution to the codebase is the reusable shape-parity comparator (`apfs_fastindex::parity`) and the env-gated integration harness that future hosts (or future macOS versions with different security policies) can rerun without code changes.

The live-disk row of the support matrix therefore reads: *not supported on stock macOS; raw read returns `EPERM` under `sudo`; the fallback walker remains authoritative for live volumes, including the privileged-subprocess invocation that unlocks TCC-restricted user-data paths.*

11.5 EX-29 Local-Snapshot Enumeration (No User Snapshots on This Host)

The product surfaces local snapshots as a row in the status bar because their bytes are part of `df`’s “used” total and the volume’s Finder-reported capacity, but the scanner walks only the live volume’s fs-tree. Without a snapshot row, those bytes look “missing” on a user’s mental model.

EX-29’s deliverable is the enumeration: a count of user-visible local snapshots, with names. Reclaimable bytes are explicitly out of scope on stock Apple silicon: there is no public read-only oracle for per-snapshot reclaimable bytes (`tmutil thinlocalsnapshots` reports the number, but it deletes the snapshot in the process), and EX-28’s verdict (`live_raw_blocked_by_kernel`) ruled out the raw extent-set diff path.

Enumeration is unprivileged. The crate’s `snapshots` module calls `tmutil listlocalsnapshots <mount>` and `diskutil apfs listSnapshots <mount>`, parses each output, and classifies the result. SR-020’s sealed-system filter excludes snapshots whose name matches `com.apple.os.update-*` (the read-only system volume’s boot snapshot, which the user can’t delete and whose bytes aren’t user-visible).

Observation (2026-05-20). The project owner’s host has zero user-visible TM local snapshots and one sealed-system OS-update snapshot on `disk3s1s1`. The classifier returned `NoUserSnapshots{sealed_system_excluded_count = 1}`. EX-23 found the same shape in 2026-05-16; on a single-user development host without Time Machine configured this is the expected steady state.

The integration harness in `crates/apfs-fastindex/tests/ex29_snapshot_enumeration.rs` mirrors EX-28’s pattern. An unconditional unprivileged test runs the classifier; a gated test exercises a raw extent diff between a mounted snapshot’s device node and the live volume, using the existing `parity::compare_namespace_shapes` comparator. The gated test reuses EX-28’s `LiveScanOutcome` classifier, so an `EPERM` on the snapshot device is recorded as “EX-28 Hypothesis C reproduces on the snapshot device” rather than failing the test.

The live-snapshot row of the support matrix therefore reads: *enumeration validated; reclaimable bytes unclaimed (no read-only oracle; raw extent diff blocked by EX-28 Hypothesis C on Apple silicon*

under SIP).

11.6 Fail-Closed Triggers

A source that does not satisfy the allowlist must fail closed before emitting rows. The categorical triggers, with the chapter that develops each:

Ambiguous or malformed checkpoint state (chapter 3).

Non-contiguous checkpoint descriptor layout (chapter 3).

Unsupported incompatible feature bits in the container or volume (chapter 3, chapter 5).

OMAP-open or OMAP-lookup encryption flags (chapter 4).

Unknown bits in any `flags` field the parser inspects (chapter 4, chapter 7).

Encryption keys or hardware-backed key requirements the parser cannot reach (chapter 4).

Fusion or multi-device storage requirements (chapter 3).

Requested merged-root or boot-root semantics (chapter 13).

Snapshot or sealed-volume semantics outside the target mode (chapter 13).

No validated corpus for the source class.

Each trigger is a typed error with a substring naming the rule. The caller dispatches on the error rather than retrying.

11.7 The Fallback

When the parser refuses a source, the product is not stuck. The fallback path emits the same shape of output through supported operating-system APIs.

Observation. A POSIX-traversal fallback in both Python (`fallback_traversal.py`) and Rust (`fallback.rs`) emits `NamespaceEntry` and `DirectoryAggregate` rows that, on the proof fixture, match the raw-mode output of the same volume exactly. Same entry count, same paths, same `entry_kind`, same `logical_size`, same `symlink_target`, same per-directory aggregates.

The fallback walker uses `os.walk` plus `os.lstat` plus `os.readlink` in the Python version, and the equivalent `read_dir` plus `symlink_metadata` plus `read_link` in the Rust version. Both paths apply the SR-017 precedence rule at the trivial step: `st_size` for regular files, `readlink` byte length for symlinks. Both apply the unique-inode aggregate policy from chapter 8.

Two consequences follow.

The contract is mode-independent. The product emits the same shape of output regardless of which backend produced it. The caller does not have to know whether the row came from a raw FS-tree walk or a POSIX traversal; the field set and the semantic mode are identical.

The slow path is not slow. Chapter 12 shows that the parallel fallback (four workers by default) sustains roughly 313 thousand entries per second on warm directory trees on Apple silicon — $2.47\times$ the post-micro-optimisation single-threaded throughput and $1.82\times$ the pre-perf baseline. On a cold whole-machine scan the bottleneck is the disk, not the parser. The fallback is fast enough to be the default for sources outside the raw allowlist, not a degraded mode.

11.8 Resilience In The Fallback

A whole-machine scan as a normal user encounters paths that the user does not have permission to read: sandboxed application data, sealed-system directories, daemon-owned caches. The fallback walker must continue across these without aborting and without producing silently incomplete totals.

Observation. Per-entry `EACCES`, `EPERM`, and `ENOENT` are recorded in `parser_output.walk_skips` with a reason; the walk continues. The same log carries the `firmlink_overlay_source` and `duplicate_dir_inode` skip kinds introduced by the dedup pass below. A normal-user / scan on the host the project tested completes with roughly five hundred recorded skips (the exact count varies with the host’s app sandbox profile and the firmlink set) and reports them in the output document. The caller can render the skip list as a diagnostic without it affecting the row stream.

The walker also skips top-level system directories that exist only as APFS-internal artefacts on the boot disk (`.fsevents`, the system snapshot mountpoints) by name, rather than letting the operating system raise an error on them. That skip is documented in the source so it can be revisited as support broadens.

11.9 Parallel Fallback

Observation. The fallback walker accepts a `-threads N` flag. The default is `min(hw.physicalcpu, 4)`; the lower bound is 1, the upper bound clamped to 4. Per-worker `BulkReader` owns its own 64 KiB `getattrlistbulk` buffer; a shared work queue distributes directories; per-worker entry vectors are concatenated and re-sorted on the main thread.

The default’s ceiling at 4 is not arbitrary. The single APFS container holds a B-tree contention point that pre-Mojave was described as a global mutex (Szorc 2018) and that the Apple Developer Forums position (Apple DTS, 2025) describes as having been largely addressed in macOS 10.14 while leaving a residual per-container cost. Empirical measurement on Apple silicon (EX-25, chapter 12) shows the throughput envelope peaking at $T = 4$ with $2.47\times$ speedup over single-threaded, then *regressing* at $T = 8$ and falling further at $T = 14$. At $T = 8$ the total system CPU is $4\times$ the $T = 1$ value while the throughput is only $1.94\times$; at $T = 14$ system CPU is $9.3\times$ for $1.38\times$ throughput — the kernel pays super-linearly more work to deliver sub-linearly more answers. The 4-thread ceiling stays clear of that regime by construction.

The parallel walker preserves the serial walker’s contract by shape, asserted by a synthetic-tree regression test (`parallel_walker_matches_serial_shape`): identical entries, aggregates, and skip lists; the only difference is the order in which directories are visited, which the final sort normalises.

Live progress events flow through a dedicated sampling thread inside the parallel scheduler at a 250 ms cadence (matching the serial path), so the CLI’s `--progress` stream and any user-space consumer of it can tick smoothly even on a whole-machine scan that is otherwise silent for tens of seconds. The in-process FFI consumed by the native shell of chapter 13 exposes the same stream through `apfs_scan_directory_with_progress`, which takes a `(*const c_void, ApfsProgressEvent*)` callback the sampling thread invokes from the library side; the SwiftUI

shell hands in a closure that updates an `@Published ProgressSnapshot` on the main actor and the toolbar’s determinate progress bar follows it. The serial-vs-parallel cadence and event payload are identical across the CLI and the FFI; one source of truth, two delivery surfaces.

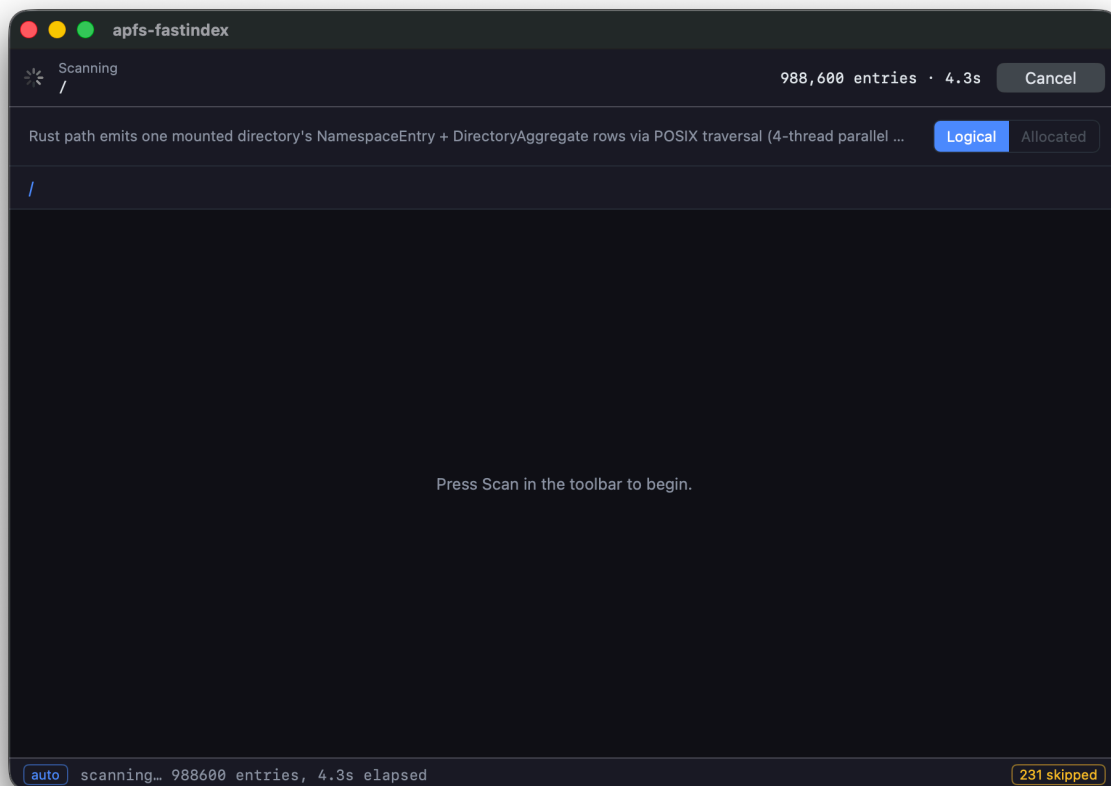


Figure 11.1: A whole-machine `/` scan mid-flight in the pre-native build that consumed `--progress` stderr events. The stopwatch and entry counter come from the parallel walker’s progress thread; 231 paths have been recorded as skipped at this point in the walk (sandboxed application data, sealed-system locations). The Cancel control terminates the worker pool cleanly. The same event stream now also flows into the native shell through `apfs_scan_directory_with_progress`, which carries the cumulative logical-bytes count as well so the SwiftUI shell’s progress bar is determinate rather than spin-wait indeterminate.

11.10 Firmlink Overlay Dedup

Observation. On macOS the user-visible startup root is a join of the System volume and the Data volume. Specific paths — `/Users`, `/Applications`, `/Library`, `/private`, `/usr/local`, and a few others — are exposed both at their canonical location *and* at `/System/Volumes/Data/<source>` via firmlinks. The same underlying inode is visible through two paths. A naive recursive walker descends both and double-counts everything inside the overlay.

The fallback walker, when the root is `/`, reads `/usr/share/firmlinks` at scan-start and builds

a `HashSet` of the `/System/Volumes/Data/<source>` overlay paths. A descent into any such path is refused with a `firmklink_overlay_source` skip; the canonical path remains the one that contributes to the namespace and aggregates.

A second layer of safety, a per-scan `Mutex<HashSet<(dev, ino)>>`, catches any directory inode reached through a path the `firmklink_overlay_source` set did not cover. A second observation of the same inode through a different path is recorded as a `duplicate_dir_inode` skip and the walker does not recurse.

The effect on whole-machine scans is large. Before the dedup, a / scan on the test host reported 5,250,000 entries; after, 3,060,000 entries with seventeen `firmklink_overlay_source` skips (one per existing `firmklink` target) and zero `duplicate_dir_inode` skips (the overlay layer caught everything). The forty-two per cent that disappeared were not real files: they were the same files counted twice through two paths.

11.11 Auto-Dispatch

The CLI binary `apfs-fastindex-scan` picks the backend automatically based on the source path's shape.

- A `.dmg` path or a `/dev/disk* /dev/rdisk*` device path is dispatched to raw mode.
- A directory path is dispatched to fallback mode.
- The caller can override with `-mode raw`, `-mode fallback`, or `-mode auto`.

The auto-dispatch is what makes the support boundary usable as a product surface. The user does not have to remember which sources are in the allowlist; the binary handles the dispatch and tells the caller, through the `correctness_claim` and `not_claimed` fields, which mode actually ran.

11.12 Why The Boundary Is Permanent

The boundary between raw and fallback is not provisional. As the allowlist grows (chapter 13 tracks the open candidates), each new source class gets its own line in the support matrix: its own allowlist row, its own oracle, its own probe. The boundary moves only when there is evidence to move it.

This means the manual will keep growing chapters of this shape: “*here is the source class, here is the oracle, here is the probe, here is the result, here is the support claim.*” The work finishes only when every source class an APFS reader is willing to claim has its own line in the matrix.

The unsexy form of the work is the price of having a fast indexer that is also correct.

Part VI

Engineering

Chapter 12

Measurement

The project’s policy on performance is straightforward: measure before optimising; record what was measured so future claims can be compared against it; never report a number whose source class is ambiguous.

This chapter contains the reproducible measurements `apfs-fastindex-scan` has produced. There are four passes: the standing baseline (chapter 11’s fallback first cut), a user-space micro-optimisation pass on the fallback walker, a parallel-walker pass that follows from the performance-lever map, and a structural-density pass against the post-FFI workload. Each pass changes one thing at a time and records the before/after numbers.

12.1 The Standing Baseline

Cold cache, normal user, release build, single host.

target	backend	entries	wall (s)	entries/s	user / sys CPU
proof fixture	raw	7	0.23	30	13 ms / 15 ms
apfs-fastindex repo	fallback (std)	9 124	0.07	130 734	15 ms / 48 ms
/Applications	fallback (std)	163 667	1.28	127 513	410 ms / 728 ms
/Applications	fallback (bulk)	163 667	1.04	157 155	347 ms / 608 ms
/Users	fallback (bulk)	1 304 073	26.65	48 933	2.92 s / 6.62 s
/ (whole machine)	fallback (std)	5 251 546	129.6	40 510	16.3 s / 47.8 s
/ (whole machine)	fallback (bulk)	5 260 624	108.7	48 380	14.85 s / 29.80 s

Reading the standing baseline:

The proof-fixture raw number is dominated by setup. Most of the 0.23 s is `hdiutil attach`; the actual seven-entry walk is sub-millisecond. The raw-mode throughput on a meaningful real volume is not yet measured because the project does not carry a large detached APFS sample. The number is honest but unrepresentative.

Fallback throughput sits at roughly 130 thousand entries per second on warm trees with `lstat` and roughly 157 thousand with `getattrlistbulk`. The improvement on `/Applications` is about 24 per cent throughput. On the cold `/ scan` the improvement is smaller: 16 per cent wall, because the disk is the bottleneck on a cold-cache run.

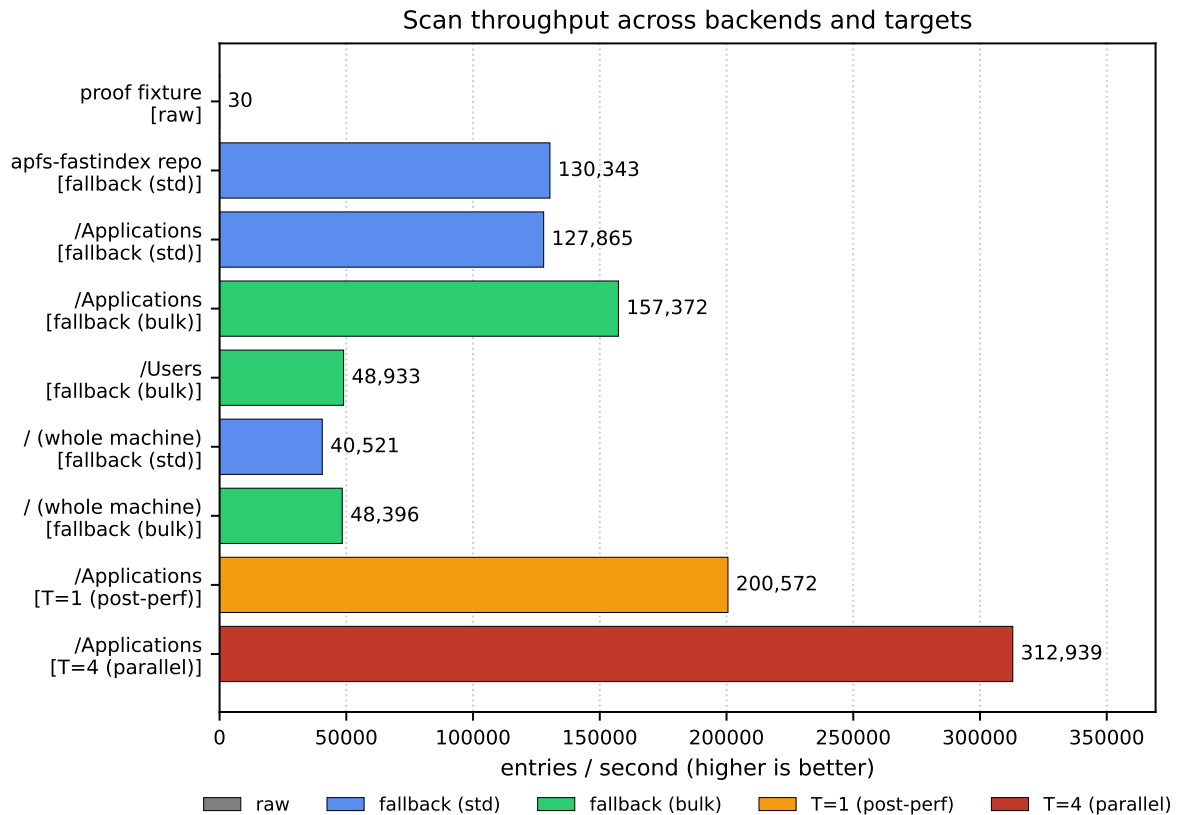


Figure 12.1: Entries per second across the measurement rows. The proof fixture is dominated by `hdiutil attach` setup; the fallback rows cluster from forty thousand to over three hundred thousand entries per second depending on backend and tree shape. The orange row is the standing single-threaded fallback after the micro-optimisation pass below; the red row is the parallel walker.

The cleaner perf signal is system CPU. On the cold `/` scan, `getattrlistbulk` drops system CPU from 48 s to 30 s — a 38 per cent reduction. Batching cuts the syscall count, and the kernel does less work per entry. The wall-time savings compound on warm-cache runs where the disk is not the constraint.

12.2 The Micro-Optimisation Pass

The standing baseline left fallback throughput on `/Applications` at roughly 172 thousand entries per second end-to-end (single-threaded, post-baseline) with system CPU dominating the budget. A user-space pass against per-entry allocations and aggregate construction landed four coupled changes:

- `BulkReader` owns one 64 KiB `getattrlistbulk` buffer and one output `Vec<BulkEntry>` across the whole walk, replacing the fresh `vec![0u8; 65_536]` the per-directory `read_directory_bulk` used to allocate.
- `build_aggregates` switched from `BTreeMap<String, BTreeMap<u64, _>` keyed by owned

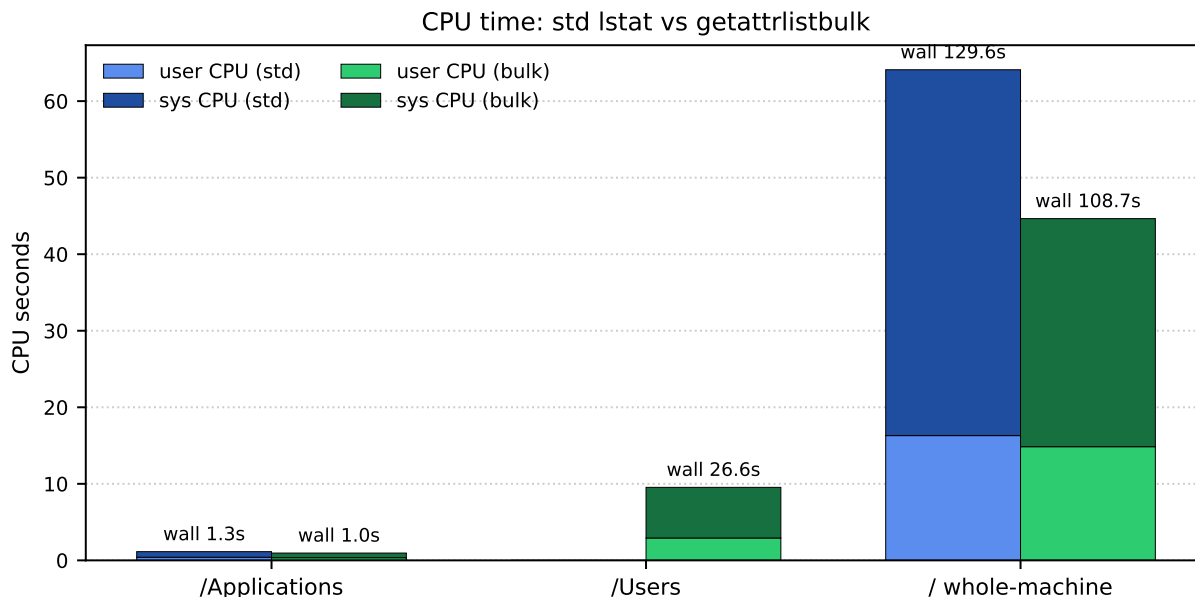


Figure 12.2: User-plus-system CPU per row, comparing ordinary `lstat` against `getattrlistbulk`. The wall-time improvement is real but modest; the system-CPU drop is the cleaner signal that bulk fetch is doing less per-entry work in the kernel.

ancestor paths to `HashMap<&str, HashMap<u64, _>` keyed by slices borrowed from each entry’s path `String`. The old shape walked ancestors as a `Vec<String>` per file — on a five-million-row / scan, twenty-five million owned-`String` allocations. The rewrite walks parents inline with `rfind('/')` and slices; the final sorted emission converts to owned `String` exactly once.

- `EntryKind` gained `Copy` (it is a four-variant unit enum), eliminating pointless `.clone()` calls in the per-entry walker hot path.
- The unconditional `relative_str.clone()` per entry was collapsed: the mount-boundary branch is the only one that ever needed a second copy; the common path moves the `String` into the `NamespaceEntry` directly. Combined with the `Copy` change above, the steady-state inner loop pushes one entry with zero heap activity beyond the `String/Vec` growth itself.

target	metric	before	after	delta
apfs-fastindex repo	median wall	58.6 ms	53.9 ms	−8%
apfs-fastindex repo	user CPU	21 ms	17 ms	−19%
apfs-fastindex repo	sys CPU	32 ms	32 ms	flat
apfs-fastindex repo	throughput	323 721 ent/s	351 757 ent/s	+9%
/Applications	median wall	950 ms	816 ms	−14%
/Applications	user CPU	340 ms	227 ms	−33%
/Applications	sys CPU	535 ms	515 ms	flat
/Applications	throughput	172 283 ent/s	200 512 ent/s	+16%

The whole-machine / scan was not re-measured here — the host’s filesystem cache state would dominate any single re-run — so the medium-tree percentages are the load-bearing data points.

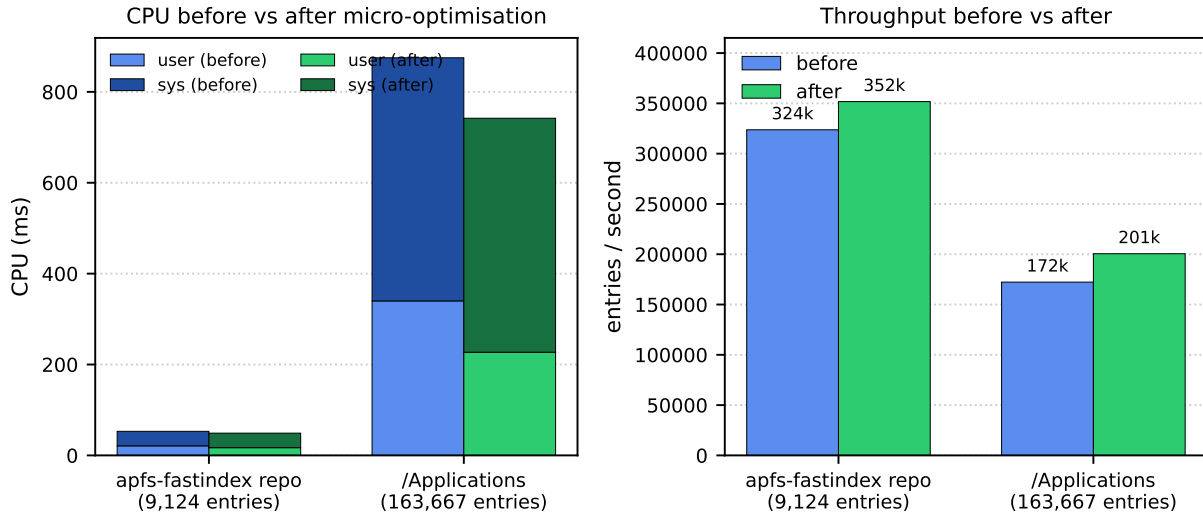


Figure 12.3: Micro-optimisation pass. The user-CPU column is the load-bearing signal: -19% on a small tree and -33% on a medium one. System CPU is flat (the syscall count did not change); only the per-directory allocator pressure is gone. The user-space savings scale with entry count because the aggregate phase dominates on larger trees.

12.3 The Performance-Lever Map

The micro-optimisation pass left the scanner at roughly 200 thousand entries per second on `/Applications` with 63 per cent of the budget in system CPU. The remaining cost was structurally kernel-side: user-space micro-opts had no further room. A source review (SR-021) cross-read four parallel research threads — xnu’s `getattrlistbulk` cost decomposition, the Spotlight alternative, an open-source-tool survey, and the `jwalk/rayon` prior art — to produce a map of which levers were structurally viable. The map flagged two candidates worth measuring (parallelism and attribute-mask shrink), two ruled-out alternatives (Spotlight, bulk buffer size), and one worth A/B testing (`fts(3)`).

Two probes ran:

EX-24: attribute-mask shrink falsification. A single-threaded microbench compared `getattrlistbulk` with three configurations on `/Applications`: drec-only (no inode-required attributes), the production mask (`ATTR_FILE_TOTALSIZE | ATTR_FILE_ALLOCSIZE`), and `fts(3)`. The drec-only and production rows landed within one per cent of each other on every dimension — wall, user CPU, system CPU, throughput — which falsifies SR-021’s hypothesis that the inode-attribute fetch path was a measurable lever on modern Apple silicon. The `fts(3)` configuration was 1.7 times slower than the production mask, contradicting Tempel’s 2019 result and cancelling the conditional follow-up. The pure kernel throughput in this microbench was 312 thousand entries per second, which gave the parallel probe a useful kernel ceiling estimate.

EX-25: parallel walker. A standalone parallel microbench swept thread counts $T \in \{1, 2, 4, 8, 14\}$ (14 being the host’s `hw.logicalcpu`). The verdict ladder required both $T = 4$ throughput $\geq 2 \times T = 1$ throughput and $T = 4$ system CPU per thread $\leq 1.2 \times T = 1$ system CPU to declare scaling validated. Both gates passed.

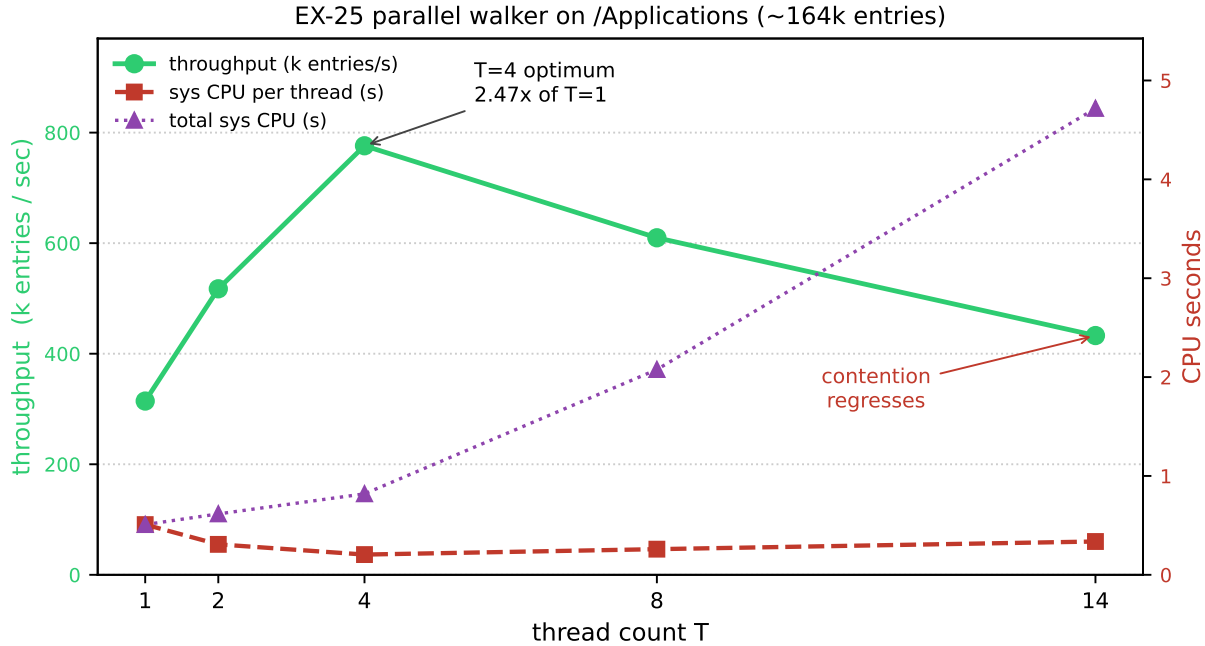


Figure 12.4: The EX-25 parallel scaling curve. Throughput rises to a peak at $T = 4$ ($2.47\times$ over $T = 1$) and then regresses; system CPU per thread stays flat through $T = 4$ and then jumps super-linearly as the APFS container contention regime fires. At $T = 8$ total system CPU is $4\times$ the $T = 1$ value for $1.94\times$ throughput; at $T = 14$ it is $9.3\times$ for $1.38\times$. The contention shape exactly reproduces the Szorc-2018 prediction on this kernel.

T	wall (s)	sys (s)	sys/T (s)	entries/s	speedup
1	0.521	0.508	0.508	314 380	1.00×
2	0.316	0.616	0.308	517 437	1.65×
4	0.211	0.819	0.205	776 196	2.47×
8	0.268	2.076	0.260	609 748	1.94×
14	0.378	4.717	0.337	432 866	1.38×

The four-thread cell is the production default. The two regression cells matter even though they are not the default: they explain *why* the default is conservative, and they tell a future maintainer how to interpret a host whose curve looks different.

12.3.1 The Parallel Slice In Product

The Rust slice landed a `-threads N` flag with default `min(hw.physicalcpu, 4)`, computed via `sysctlbyname("hw.physicalcpu")` with `std::thread::available_parallelism()` as the cross-platform fallback. The walker’s contract is preserved by shape: per-worker `BulkReader`, shared `WorkQueue` (mutex plus condvar plus outstanding counter plus an atomic done flag), per-worker entry vectors concatenated on the main thread, the existing `sort_unstable_by` plus `build_aggregates` normalising ordering. A synthetic-tree regression test (`parallel_walker_matches_serial_shape`) asserts byte-identical entries, aggregates, and walk-skip lists across the two schedulers.

End-to-end measurement on `/Applications`, same host as the previous tables, median of five runs each side:

metric	$T = 1$ (post-pass)	$T = 4$ (parallel)	delta
median wall	816 ms	523 ms	−36%
throughput	200 512 ent/s	312 879 ent/s	+56%
user CPU	227 ms	256 ms	+13% (worker setup)
sys CPU	515 ms	802 ms	+56% (kernel scales)

Cumulative `/Applications` throughput across both perf passes: $172\,283 \rightarrow 312\,879$ entries per second, +82% ($1.82\times$). The user-CPU gap between the production scanner (256 ms at $T = 4$) and the parallel microbench (about 20 ms at $T = 4$) quantifies the user-space post-processing tax — `NamespaceEntry` allocation, per-worker merge, final sort, aggregate construction. It is recorded as a candidate future optimisation; the structural-density pass below is the work that followed.

12.4 The Structural-Density Pass

The native-shell rewrite of chapter 13 introduced a new dominant workload: the FFI scan plus indexed-tree build that the SwiftUI app performs on every `Choose Folder`. The first measurements against this path on a representative real tree (`/Users/kai/Projects`, 320 000 entries, four threads, warm cache) showed where the post-rewrite cost actually sits: roughly 600 milliseconds of scan wall, 68 milliseconds of tree build, and 262 mebibytes of peak resident set. The walk-time work the previous pass had already taken; what was left was structural — the shape of the in-memory representations and the work the build phase did once the bytes were back.

A bench-driven pass (eight rounds, each backed by per-phase timings the `APFS_PHASE_TIMINGS=1` envvar surfaces and allocator-pressure counts the `examples/perf_probe` binary records) attacked that surface change by change. Each round changes one thing; each round is recorded in the commit that landed it (`git log --grep='perf('`); the cumulative shape across all eight is the table below.

metric	before	after	delta
scan wall ($T = 4$, warm)	~600 ms	~410 ms	−32% ($1.5\times$)
tree wall (warm)	68 ms	34 ms	−50% ($2\times$)
peak RSS	262 MiB	190 MiB	−28%
<code>TreeNode</code>	168 B	112 B	−33%
<code>NamespaceEntry</code>	88 B	72 B	−18%
scan-time allocations	2.23 M	2.09 M	−7%
scan-time bytes	580 MiB	330 MiB	−43%

Most of the eight rounds are mechanical: the structures the scan produces and the indexed tree reads once per scan settle into a narrower shape, with one fewer indirection per access wherever an interior allocation could be folded into a contiguous arena or a borrowed slice. Three are worth naming separately because each implicates a larger principle.

Box<str> over String for write-once fields. Both the flat `NamespaceEntry` and the indexed `TreeNode` carry strings that are built once during the walk or the index build and never mutated. The eight bytes of capacity each `String` carries against the day it might grow is wasted work on a

write-once row. Switching `path`, `symlink_target`, and `name` fields to `Box<str>` shaves twenty-four bytes per `TreeNode` across three fields and sixteen bytes per `NamespaceEntry` across two, without changing the build side: the strings already came from `into_boxed_str()` on a fresh `String` whose capacity equalled its length, so the shrink is a metadata-only operation. (Rounds 1 and 2.)

Non-cryptographic hashing for non-adversarial keys. Two of the build’s largest `HashMap`s — the `Tree::build` per-directory child map keyed by short filenames, and the `build_aggregates` outer map keyed by ancestor-path slices and inner map keyed by inode numbers — were running on `std`’s default `SipHash`, which trades collision-resistance against speed. Filenames and inode numbers are not adversarial input; the collision-resistance trade is moot. Swapping the hasher to `rustc_hash::FxBuildHasher` dropped `Tree::build` from sixty-three to forty-seven milliseconds (round 3, -25%) and `build_aggregates` from roughly 150 to 91 milliseconds (round 7, -40%) on the same input. The change is bounded to build-time maps; the read side is untouched. (Rounds 3 and 7.)

Don’t compute what the caller does not read. The `DirectoryAggregate` table the fallback produces is a flat `Vec` keyed by directory path with per-inode contribution totals. The CLI’s JSON output consumes it; the SwiftUI FFI does not (the indexed tree’s `value_logical` and `value_allocated` cover the same surface from a different angle). A boolean option on `FallbackOptions::skip_aggregates` — false by default, true on the FFI path — skips the build entirely. The change saved roughly ninety milliseconds of scan wall, 250 mebibytes of scan-time allocator pressure, and roughly thirty-four mebibytes of peak resident set on the FFI path; the CLI’s JSON shape is unchanged. (Round 8.)

The other five rounds (drop `TreeNode.path` and lazy- compute it from the parent chain; flatten per-directory `Vec<u32>` children into a single contiguous arena; shard `VisitedDirs` into sixteen mutexes keyed by a hashed (`dev`, `ino`); pre-size per-directory child vectors; `Box<str>` for the inflight child-map keys so the map and the `TreeNode.name` share one allocation) follow the same shape: identify the wasted indirection, the wasted capacity, or the wasted lock, and remove it.

Two more candidates were probed and falsified inside the pass and warrant naming so they are not relitigated. The `getattrlistbulk` buffer size is not a perf lever on this workload (every value tested from 32 KiB through 2 MiB landed within noise; 4 MiB regressed), exposed at runtime via the `APFS_BULK_BUF_KIB` envvar for any future re-ablation. And EX-24’s kernel-side falsification of the attribute-mask shrink hypothesis means the next bounded lever on the FFI path is no longer in user-space allocation or hashing — it is the walk itself, where the kernel is already doing the syscall work `getattrlistbulk` reduced.

12.5 What The Numbers Do Not Say

The table is a baseline; it is not a claim about anything beyond what it measures. Three categories are explicitly outside it:

1. Raw-mode throughput on a real source. The project has not measured raw mode on a representative live or detached volume of meaningful size. Until it does, the comparison “raw vs fallback” cannot be made.
2. Incremental scans. The current model is full reparse on every run; the cache work of chapter 10 has not landed in product. Repeat-scan numbers will only mean something after a cache implementation passes its own oracle.

3. Encrypted or live-boot-disk surfaces. The parallel walker has not been measured on encrypted APFS; per SR-021's review of Petrenko (2020), the surviving contention may behave differently there.

12.6 The Measurement Discipline

Three habits keep the numbers honest.

Measure before optimising. Each pass landed only after a microbench established what the lever was worth. The micro-optimisation pass was justified by an allocator-pressure profile; the parallel pass was justified by EX-25's scaling verdict; the attribute-mask refactor was killed by EX-24's falsification of the vnode-rage cost.

Record the source class and the semantic mode with each number. The tables above carry both. A throughput number without those is not interpretable: 300 thousand entries per second is either impressive or unimpressive depending on whether it ran cold or warm, against raw or fallback, single-threaded or parallel, against `/Applications` or `/`.

Hold the baseline. Any future performance claim has to compare against the tables above or against a row added with the same shape. A new optimisation that does not improve a baseline row is not yet evidence of an improvement; it is a hypothesis awaiting measurement. The discipline that keeps the tables honest is the same discipline that keeps the manual honest.

Chapter 13

Open Questions

The previous chapters develop the manual’s defended scope: one volume, one pinned state, namespace plus logical size plus the allocated-size column where it has an oracle, fallback for everything else. This chapter is everything that sits on the boundary — the work that has been done since the foundation settled, and the work that has been scoped but not yet validated. A reader who wants to understand the surface of unknowns around an APFS reader, rather than the well-explored interior, reads this chapter.

Each section below names one open question, what is known about it, what is not, and the smallest probe that would close it.

13.1 Allocated Size

The most visible WizTree-equivalent metric, and the one whose on-disk story is least settled.

The question. For each inode, what number of bytes does APFS allocate to its content on disk?

The result. Every case class the EX-26 corpus exercises emits a per-inode `allocated_size` that matches macOS’s `st_blocks * 512` oracle exactly. Sparse files emit `allocated_size - sparse_bytes` (EX-26 Hypothesis A, validated across four sparse shapes after EX-22 first observed the identity on one). Decmpfs-compressed files emit the sum of any stream-backed xattr dstreams’ allocated bytes (EX-26 Hypothesis F, validated across both ditto-produced shapes: xattr-stream-stored and resource-fork-stored). Chapter 8 gives the precedence rule, the case-class table, and the per-row evidence.

What is still open. Exclusive bytes (bytes only this inode owns, after accounting for clones and snapshots) and shared bytes (bytes shared across inodes or snapshots) require the extent-reference tree to be opened and joined against per-inode extents. Neither has been validated end to end against a public oracle; both are separate questions with their own corpora. EX-27 is the planned probe for clone-deduplicated allocated bytes via the extent-reference tree.

13.2 Snapshot-Assisted Scanning

The mechanism by which a fast indexer can scan a live machine honestly. Entitlement-blocked rather than technically blocked: the walker is wired, but the validation oracle requires privileges the project’s reproducer host does not have.

The question. If the indexer takes a local APFS snapshot of a live volume, mounts it read-only, and walks it through the existing fallback, does the result match what the same walker would

have emitted on the live volume at the snapshot moment?

What landed. A source review of the snapshot API surface (SR-020) made the privilege boundary explicit: `fs_snapshot_create(2)` requires root *and* the `com.apple.developer.vfs.snapshot` entitlement; the same gate applies to delete, rename, mount, and revert. `mount_apfs -s` also requires root. `tmutil localsnapshot` is the only unprivileged create path; it invokes `backupd` over XPC and only works on volumes included in the Time Machine backup set. `fs_snapshot_list(2)` and the corresponding `diskutil` and `tmutil list` subcommands are free.

The scanner therefore cannot stage its own snapshot inside an unprivileged test process. The Rust integration — a `-snapshot <mountpoint>` flag that consumes an already-mounted snapshot directory and runs the existing fallback against it — is operationally useful immediately. The shape-parity claim itself is parked behind an unblock condition: the EX-23 probe ran on a clean dev workstation, found no user-visible snapshots (only the sealed-system `com.apple.os.update-*` snapshot, which it correctly skipped), and emitted a `blocked_no_snapshots_at_all` verdict with the exact `sudo mount_apfs -s` reproducer command saved in the probe artefacts.

What is still open. The shape-parity oracle itself. Until a privileged rerun produces a positive verdict, the `-snapshot` flag tags the source as a snapshot in `correctness_claim` and `not_claimed` but does not promise that the result matches a live-directory walk on this host. The `not_claimed` register names the missing oracle explicitly so a consumer cannot accidentally treat a snapshot scan as equivalent to a live scan.

The smallest probe that would close it. A privileged rerun. The reproducer is:

```

1 tmutil localsnapshot /
2 tmutil listlocalsnapshots /
3 sudo mkdir -p /tmp/apfsfi-ex23-snap
4 sudo mount_apfs -s <snapshot-name> / /tmp/apfsfi-ex23-snap
5 python3 docs/research/experiments/EX-23-snapshot-shape-parity/artifacts/probe_ex23.py
6 sudo umount /tmp/apfsfi-ex23-snap

```

A positive verdict would unlock moving the snapshot-mount cell of the support matrix from `not_claimed` to `fallback_supported`.

13.3 Scanner Ergonomics and Performance

The questions in this section do not change what the parser claims about APFS, only how the claim is delivered. They are worth recording because the answers — which performance levers matter on macOS-on-Apple-silicon and which are folklore — are themselves contributions to the literature on reading APFS at speed.

The questions. Which parts of the scanner’s user-facing surface — backend selection, progress reporting, embeddability in a native UI, attribute fetch cost, parallelism — can be improved without moving the emission contract?

The performance-lever map. A source review (SR-021) synthesised four parallel research threads (xnu source, the Spotlight alternative, an OSS-tool survey, and the `jwalk/rayon` prior art) into one map.

Falsified. EX-24 measured the attribute-mask shrink lever: drec-only attributes (no inode-required fields) versus the production walker’s attribute mask (`ATTR_FILE_TOTALSIZE |`

`ATTR_FILE_ALLOCSIZE`). The two rows landed within one per cent of each other. The SR-021 hypothesis that the inode-attribute fetch path crosses xnu’s vnode-rage boundary as a measurable lever is *falsified for modern Apple silicon and this attribute set*. The Phase-2 deferred-attribute refactor is therefore de-prioritised. The `fts(3)` control was $1.7\times$ slower than the production mask, contradicting Tempel’s 2019 result and cancelling the conditional follow-up.

Validated. EX-25 measured the parallel-walker lever at $T \in \{1, 2, 4, 8, 14\}$. $T=4$ reached $2.47\times$ the single-threaded throughput; the system-CPU-per-thread cost stayed flat through $T=4$ then jumped super-linearly. Chapter 12 gives the curve. The parallel slice landed with default `min(hw.physicalcpu, 4)` and a synthetic-tree regression test asserting byte-identical output across schedulers.

Ruled out. Spotlight as a metadata oracle (by-design incomplete coverage, no scoped-subtree streaming API, no row-formatted output, undocumented store format, write visibility lag). `getattrlistbulk` buffer size larger than 64 KiB (the per-syscall floor is sub-0.1% of the budget). `getdirentiesattr(2)` (deprecated since macOS 10.10, routes through the same VFS path on APFS).

Other improvements that landed. A firmlink-overlay dedup that cut whole-machine / scans from 5.25M to 3.06M entries by refusing the `/System/Volumes/Data/*` duplicate paths (chapter 11); a progress-cadence change from one second to 250 ms so the stopwatch on a fast CLI scan does not freeze; and the native shell developed in the next subsection, which moved the visualisation from a WebKit-embedded JS treemap to an in-process Rust-and-Core-Graphics renderer.

13.3.1 The Native Shell

The lane’s product surface is a SwiftUI app whose visualisation is laid out by Rust and drawn by Core Graphics. The scanner is not a subprocess; the Rust crate is linked into the app process as a static library, and every call from Swift into the indexer crosses a `cbindgen`-generated C ABI. There is no JSON serialisation, no temp file, no WebKit, and no JavaScript.

Why the rewrite happened. The pre-rewrite pipeline emitted JSON from a subprocess, wrote it to a temp file, served it through a `WKURLSchemeHandler`, parsed it in WebKit, built a hierarchy in JavaScript, laid it out with `d3.treemap`, and drew it into a `<canvas>`. A series of optimisations — canvas instead of SVG, bytes-aware ingest, hot-loop rewrites of `buildHierarchy`, lazy hit-grid construction — pulled `/Applications`-class scans into the one-second range. On a whole-machine scan the JS path still spent five-to-eight seconds between the scanner exit and the first painted rectangle. A separate probe (documented in `docs/implementation/viz-perf-study.md`) falsified the most attractive remaining JS lever (a pure-JS `msgpack` decoder, which V8’s `JSON.parse` beat by about three times) and projected that going native would collapse the same window to roughly six hundred milliseconds to one and a half seconds. The projection was the justification for the rewrite; the rewrite landed and the projection held.

The FFI boundary. The crate’s C ABI is a small surface of opaque-handle constructors, accessors, and destructors. `apfs_scan_directory` returns an `ApfsScan*`; the handle owns the entry vector, the indexed tree, and the strings the Swift side reads through `String(cString:)`. `apfs_layout_new` returns an `ApfsLayout*` whose flat `ApfsCell` array Swift reads as an

`UnsafeBufferPointer<ApfsCell>` — no per-cell call into Rust during paint, and no array copy across the language boundary. Bulk data is handed across as `(*const Item, usize)` pairs; per-handle `_free` functions are responsible for releasing every allocation `_new` produced. The header is regenerated by `cbindgen` on every change to `src/ffi.rs` so the two sides cannot drift in the type signatures they agree on.

The division of labour. Rust owns: the fallback walker, the firmlink dedup, the indexed tree (one `TreeNode` per directory and per leaf, parent and children indices, pre-computed `value_logical` / `value_allocated` / `item_count` aggregates), the squarified treemap layout (a direct port of `d3-hierarchy`'s `treemapSquarify`), the 64×64 hit-grid, the per-extension aggregator behind the right-hand panel, and the colour palette that decides a leaf's `fill_rgb`. Swift owns: the SwiftUI shell (toolbar, breadcrumb, status bar, depth stepper, metric picker), the AppKit treemap view (one `NSView` whose `draw(_:)` walks the cell array as one `fillRect` per cell, grouped by colour to amortise `setFillColor` calls), the tree-list and ext-list panels, and the right-click menu plumbing onto `NSWorkspace` for the `Open`, `Reveal in Finder`, `Copy Path`, and `Move to Trash` actions. The renderer touches no JavaScript and no DOM; the shell touches no APFS bytes.

What got removed. The repository still ships a standalone `viz/index.html` for drop-a-file-into-browser use (a separate workflow that does not require the native build), but the app itself no longer embeds it. With the WebKit dependency gone, several layers of intermediate representation went with it: the JSON serialiser as a post-scan step, the `rmp-serde` msgpack encoder, the JavaScript `buildHierarchy` hot loop, the `d3.treemap.squarify` call site, the `<canvas>` DOM node, the JavaScript hit-grid, the `WKURLSchemeHandler`-via-`ArrayBuffer` ingest path, and the JS-to-Swift bridge protocol that carried the typed messages between them. The scanner subprocess is gone; so is the post-scan ingestion gap that motivated the loading overlay an earlier slice of the lane introduced.

Progress and polish. The FFI exposes a `apfs_scan_directory_with_progress` entry point that takes a C-ABI callback the library's sampling thread invokes at the same 250 ms cadence the CLI's `--progress` stream uses; the callback's `ApfsProgressEvent` carries (`phase`, `scanned`, `skipped`, `bytes`, `terminal`) so the SwiftUI shell can render a *determinate* progress island (phase label, scanned/skipped counts, cumulative bytes against `volume.used`, m:ss stopwatch). The standard macOS settings scene (depth and worker-thread `@AppStorage` preferences), the right-click context menu, the breadcrumb-and- back-nav header, the depth-limited progressive treemap (which defers the deepest level of cells off the first render so the initial paint lands tens of milliseconds sooner on a whole- machine scan), and the unified Trebuchet MS / monospaced-digit typography all landed alongside the FFI progress surface; they are recorded here because each is a small visible move that the single chapter-13 narrative would otherwise lose.

What the screenshots show. The figures below were captured against an earlier build of the SwiftUI shell, before the tree-list and ext-list side panels, the breadcrumb bar, the determinate progress island, and the settings scene landed. The toolbar and the treemap content read the same in both builds because they describe the same underlying data; the current build adds two resizable side panels above the treemap, the breadcrumb above them, and the centered determinate progress island during a scan.

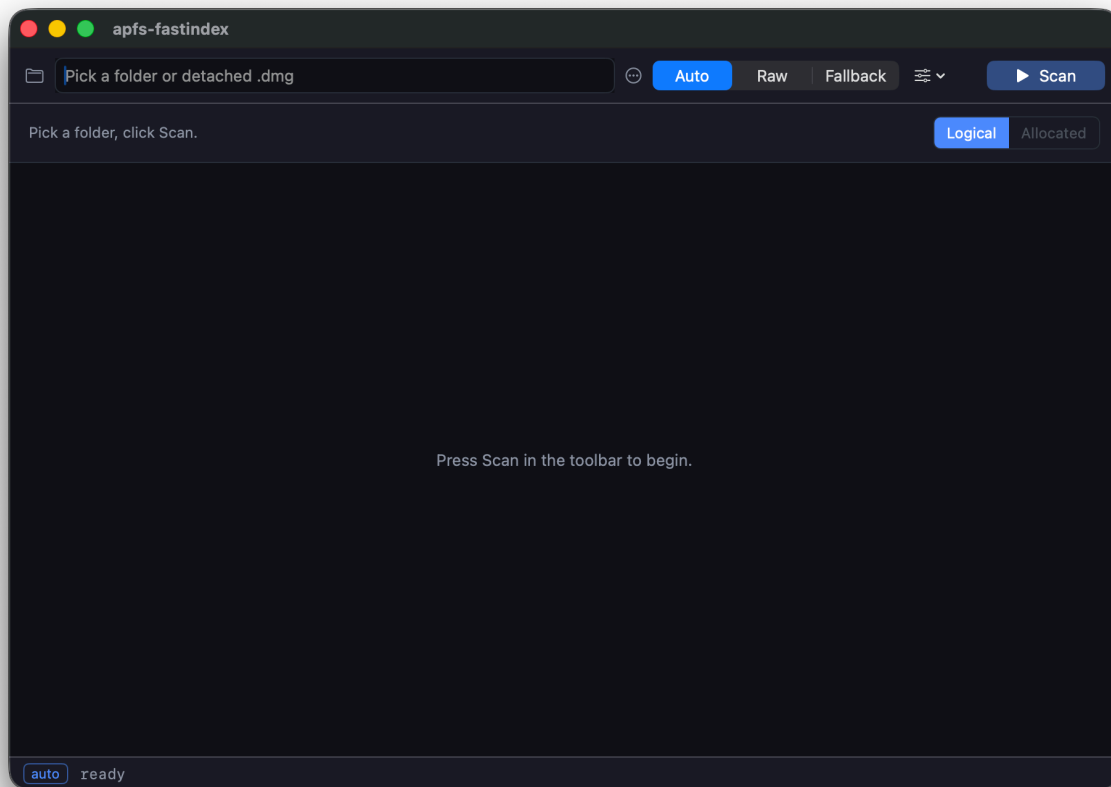


Figure 13.1: The shell in its idle state: source picker, backend mode (**Auto** / **Raw** / **Fallback**), metric picker (greyed until a scan completes), and the **auto** / **ready** status. No scan has been requested yet.

Future direction. The user-space post-processing tax recorded as “the next bounded scanner-side lever” in chapter 12’s parallel-walker section was the input to the structural-density pass the same chapter now records. The pass landed eight rounds against the FFI workload — -32% scan wall, -50% tree wall, -28% peak RSS, with allocator pressure roughly halved (chapter 12, § Structural-Density Pass). What remains is no longer user-space allocation or hashing on this workload; the next bounded lever moves back to the kernel side, where EX-24 already falsified the most attractive structural candidate. A second oracle on a larger or different-shaped tree (a /-scale scan on a freshly cleared filesystem cache; a tree with deeper-than-usual nesting; a case-insensitive APFS volume) is the cheap follow-up before any further user-space rounds are planned. The progress-FFI gap this paragraph used to record is closed: `apfs_scan_directory_with_progress` now carries the same event stream the CLI’s `--progress` flag does, the SwiftUI shell renders it as a determinate progress island, and the CLI path is unchanged for headless use.

13.3.2 The Treemap Layout

The visualisation’s correctness is now a Rust correctness question. Three pieces are worth fixing in this manual; each is small, but each is the kind of detail that decides whether two pictures of the

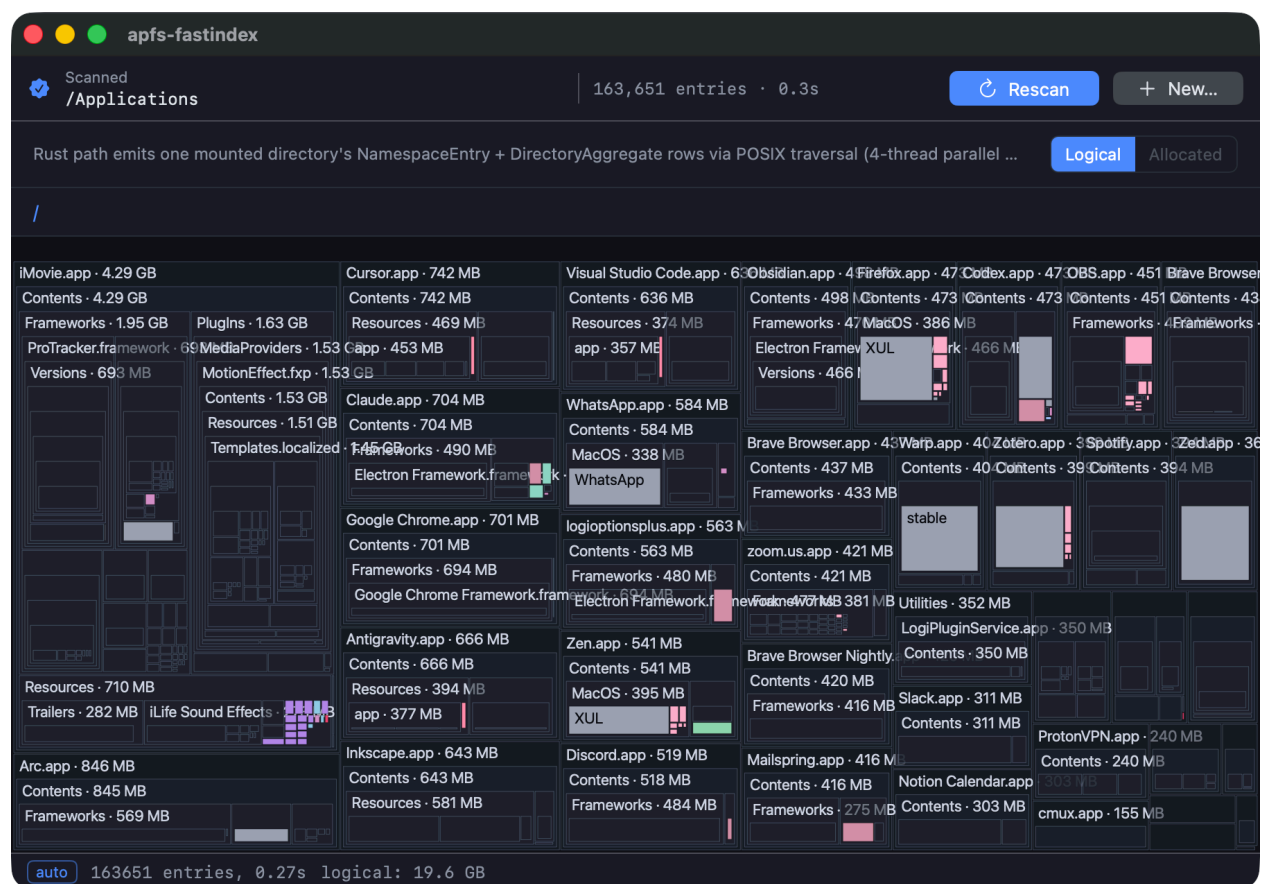


Figure 13.2: A loaded scan of `/Applications`: 163,651 entries laid out in 0.3 s by the four-thread parallel fallback. Leaf rectangles carry path and size; the colour family encodes file kind. The status bar reads the total logical bytes for the visible aggregate. In the current native build the rectangles are drawn by Core Graphics from a cell array Rust laid out directly; the visible result is unchanged.

same data agree.

Squarify. The layout is the standard squarified treemap of Bruls, Huijzen, and van Wijk (2000), in the variant `d3-hierarchy` implements. Children are sorted descending by the active metric; greedy row-packing on the shorter side of the current rect adds children to a row until the worst-case aspect ratio would worsen, then lays the row out by dice (horizontal) or slice (vertical) and recurses on the remaining strip. The Rust port keeps the same $\alpha = \max(dy/dx, dx/dy) / (\text{value} \cdot \text{ratio})$ and $\beta = \text{sum_value}^2 \cdot \alpha$ expressions the d3 source uses; for the same input (`node_index`, `value`) array and the same viewport, the two implementations produce the same rectangles.

Hit-grid. A 64×64 axis-aligned spatial hash sits over the laid-out cell array. Each cell is inserted into every bucket it overlaps; each bucket is then sorted depth-descending so the first containing match a hit-test walk sees is the deepest cell at that point. This is what makes a mouse-move event a few-cell comparison rather than a walk through the hundred-thousand-cell layout for a `/`-scan view.

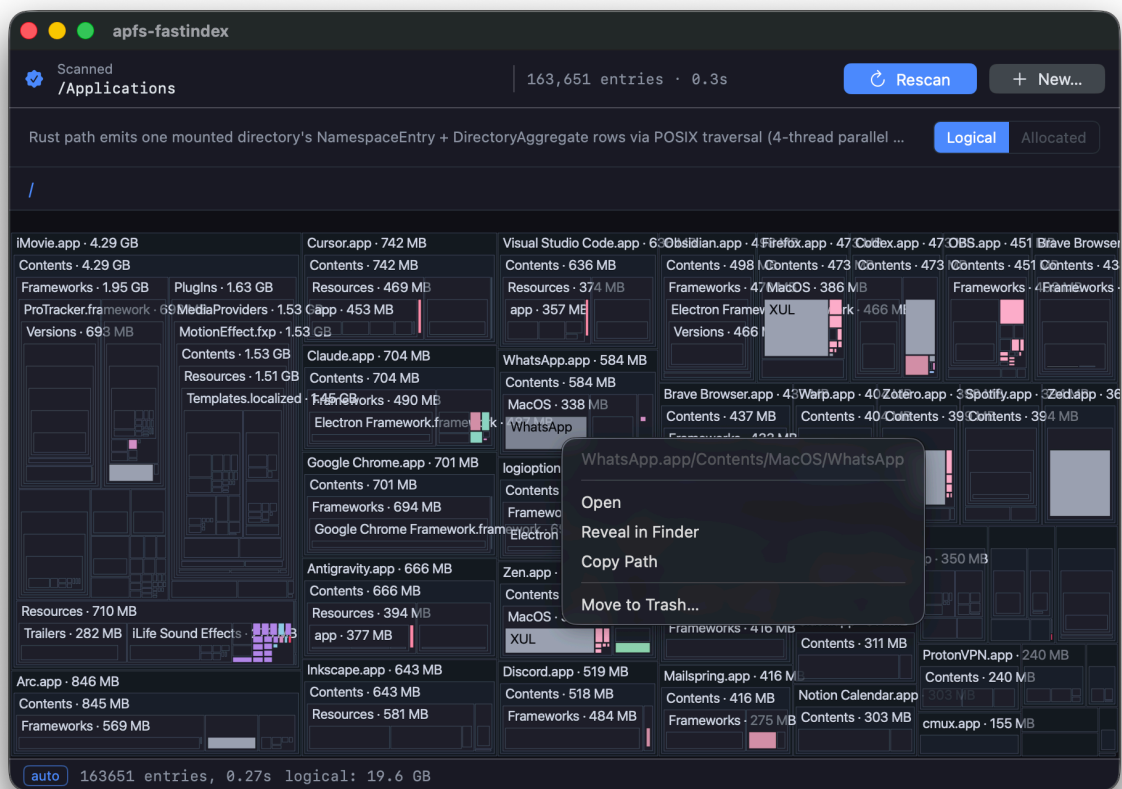


Figure 13.3: A right-click context menu over a leaf (`WhatsApp.app/Contents/MacOS/WhatsApp`). The menu is a native `NSMenu`, positioned at the click event the treemap view forwarded through the SwiftUI coordinator. `Open`, `Reveal in Finder`, `Copy Path`, and `Move to Trash` are real `NSWorkspace` calls; they inherit the operating system’s permissions, undo semantics, and recovery behaviour.

The unclaimed swatch. When the metric picker is on `Allocated` and a leaf’s `value_allocated` is `None` (SR-019 None-collapse, chapter 8), the cell’s `fill_rgb` is the unclaimed grey rather than a colour from the extension palette. The rule is one branch at layout time, but it is the visual surface of the fail-closed discipline: the user sees, on the picture, exactly the cells the parser refused to claim a number for.

13.4 Deliberately Deferred

Four APFS surfaces remain deliberately outside the manual’s defended scope. Each is large enough to warrant its own oracle work; each is unsafe to open opportunistically.

Encryption. FileVault-encrypted volumes and hardware-backed encryption have separate threat models, separate privilege requirements, and separate validation problems. The parser currently fails closed on encryption flags; opening encryption support requires deciding which classes of key

access an APFS reader is willing to use and which it is not.

Live boot disk. The startup disk on modern macOS is a sealed system volume joined to a data volume by firmlinks, with a snapshot pinning the system content. Raw scanning of the System volume is constrained by the seal; raw scanning of the Data volume is constrained by the live-churn problem; the merged user-visible view depends on firmlink table semantics that are themselves not fully public. Opening live boot support means building each of those subsystems' oracle.

Boot-root merged namespace. The Finder-visible / on a modern Mac is not any single APFS volume's namespace. It is the System volume's content joined to the Data volume's content through firmlinks, plus the current snapshot overlay. The reader mode for this is distinct from raw-single-volume; it cannot share the same oracle, and its support claims must be stated separately. The firmlink-dedup slice in chapter 11 is the closest this work has come to it: the on-disk duplication is suppressed at the fallback layer, without any claim on Finder semantics proper.

Persistent incremental cache. The work of chapter 10 turns into a product only after the full set of cache-invalidation conditions has been enumerated, a crash-safe cache storage design exists, and an incremental scan has been validated against a fresh full reparse. The questions above deliberately precede this work; a cache built on top of incomplete metrics or unstable support matrices would propagate its instability.

13.5 The Manual's Own Future

The chapter ends where the manual ends. Each question above will, when its oracle work lands, become its own chapter: a thesis, the evidence, the precedence rule or the oracle pairing, the per-case table, the limits of what was checked. The chapter list will lengthen; the voice should not change.

The supported interior of the manual is narrow but defensible. The open surface around it is genuinely open: a metric whose fail-closed cases are honest, a snapshot path correctly entitlement-gated, a perf envelope empirically calibrated against a contention regime the project did not invent and does not pretend does not exist, a native renderer whose correctness is now a Rust-side question rather than a JavaScript-side one. Anything written here that survives the next year of work survives because it was written carefully. Anything that does not survive was an error worth recording so the next attempt does not repeat it.

Glossary

The vocabulary the manual uses, in alphabetical order. Terms that appear in code identifiers carry the typewriter font of the identifier as well as the prose definition.

Allocated size. Bytes the filesystem has assigned to a file’s content. Distinct from logical size (the user-visible byte length) and from exclusive size (bytes only this inode owns). Validated for ordinary, clone, hard-link, symlink, and directory case classes; fail-closed for sparse files and decmpfs-compressed files (chapter 8). Emitted as `allocated_size: Option<u64>`; `None` indicates a deliberately-not-claimed row.

allocated_size. The `j_dstream_t.allocated_size` on-disk field. The source for the allocated-size column on the case classes that emit a value. On macOS-produced volumes the field equals `round_up(ds_size, blocksize)`; on sparse files this overstates the actual allocation by the sparse-bytes hint.

APSB. APFS volume superblock (`apfs_superblock_t`). Reached through the container OMAP at the scan XID; carries the volume OMAP, the FS-tree root, feature masks, role, UUID, and statistics.

Block zero. The fixed locator block at the start of an APFS container. Carries enough metadata to find the checkpoint descriptor area but is not itself authoritative for the live container state.

ApfsCell. The flat, `#[repr(C)]` record the Rust renderer hands across the FFI as a contiguous array. Holds the cell’s rectangle, depth, the index of the tree node it represents, a flag bitset (`is_dir`, `is_symlink`, label-strip reserved), and a precomputed `fill_rgb`. Read by Swift as an `UnsafeBufferPointer<ApfsCell>` with no array copy.

ApfsLayout. The Rust handle that owns the laid-out `ApfsCell` array and the spatial-hash index used for hit-testing. Constructed by `apfs_layout_new`, freed by `apfs_layout_free`. One `ApfsLayout` represents one (`subtree-root`, `depth`, `metric`, `width×height`) configuration.

ApfsScan. The Rust opaque handle the C ABI returns from `apfs_scan_directory` (or `apfs_scan_directory_with_progress`, which adds a sampled progress callback at the same 250 ms cadence as the CLI’s `--progress` stream and carries a cumulative `bytes` field so the SwiftUI shell’s progress bar is determinate). Owns the `FallbackScanOutput` (entries, walk-skips, `correctness_claim`, source descriptor), the indexed tree built from it, and a path-string cache the `apfs_scan_node_path` accessor populates on first query (chapter 12, structural-density pass round 4). Released by `apfs_scan_free`.

BulkReader. The Rust struct that owns a reusable 64 KiB `getattrlistbulk` buffer and one output `Vec<BulkEntry>` across the whole walk. The micro-optimisation pass introduced it to replace the per-directory fresh allocation.

cbindgen. The Rust crate that walks `src/ffi.rs` and emits the C header Swift imports. Run by the `build.rs` on every change so the C declarations cannot drift from the Rust `#[no_mangle] extern "C"` surface. The generated header lives in `target/<profile>/include/apfs_fastindex.h`.

Checkpoint descriptor area. A contiguous run of blocks that hold the rotating ring of container superblocks and their associated checkpoint maps. The authoritative live NXSB lives here.

Checkpoint map. A block carrying ephemeral-OID mappings for the selected checkpoint. Walked in order from `nx_xp_desc_index`; the last block carries the `CHECKPOINT_MAP_LAST` flag.

decmpfs. The Apple compression xattr family. Small compressed files store their data inline in `com.apple.decmpfs` with no `j_dstream_t`; larger files use `com.apple.ResourceFork`. The parser fails closed on the allocated-size column for any inode whose only size source is decmpfs, because no public reader has been oracle-validated against the macOS `st_blocks * 512` for the resource-fork path.

Directory aggregate. A per-directory total expressed under the unique-inode logical-sum policy: each inode contributes its logical size at most once per aggregate root, regardless of how many hard-linked paths inside that root name it. Allocated-size aggregates use the same rule plus a `None`-propagation step.

DIR_REC. Directory record. The FS-tree leaf that represents a directory entry; key carries the parent inode's file ID and the name; value carries the child file ID and the POSIX entry type.

Dstream. An APFS data stream. Logical and allocated size plus a default-extent reference; carried inline in the inode's xfields as `INO_EXT_TYPE_DSTREAM` or in a separate `j_dstream_id_val_t` record.

Ephemeral object. An APFS object that lives in memory between transactions and is written into the checkpoint data area at commit. The checkpoint map identifies it by `(oid, paddr)`.

Fail closed. The project's discipline of refusing unknown, malformed, or out-of-allowlist input with a typed reason, rather than producing a partial or best-effort result.

Fallback path. The supported-API backend that runs when raw mode rejects a source. POSIX walk plus `lsstat` plus optional `getattrlistbulk`; emits the same row shape as raw mode.

Firmlink. A macOS presentation mechanism that joins selected paths from a System volume to corresponding paths on a Data volume to produce the user-visible startup namespace. Out of scope for raw mode. The fallback walker reads `/usr/share/firmlinks` at scan-start and skips the `/System/Volumes/Data/<source>` overlay paths so the canonical path is the one that contributes to namespace and aggregates.

FsRecordDump / FsRecordRow. The Rust crate's structured output for the FS-tree record-family dump and one decoded record respectively. Field-level equal to the independent Python decoder on the proof fixture.

FS tree. The volume-level B-tree of variable-length records keyed by (`object_id`, `type`). Holds directory entries, inodes, xattrs, sibling-link records, dstream records, and extents.

getattrlistbulk. macOS syscall that returns metadata for many directory entries per call. Used by the fallback walker for the bulk perf path. Adding inode-required attributes (size, mode, mtime) to the attribute mask used to be thought a load-bearing cost; EX-24 falsified that for modern Apple silicon.

Hit-grid. A 64×64 axis-aligned spatial hash sitting over the `ApfsCell` array. Each cell is inserted into every bucket it overlaps; each bucket is then sorted depth-descending so the first containing match a hit-test walk sees is the deepest cell at that point. Constant-time lookup turns mouse-move into a few-cell comparison even on a /-scan layout.

Hard link. A directory entry that names an inode already named by another directory entry. APFS represents the relationship through `SIBLING_LINK` and `SIBLING_MAP` records; the unique-inode aggregate policy ensures the inode's logical size is counted once per aggregate root.

Indexed tree. The in-memory `Tree` the FFI builds once per scan from the flat `Vec<NamespaceEntry>`. One `TreeNode` per directory and per leaf, 112 B per slot post-structural-density-pass, carrying name (as `Box<str>`), parent index, a (`children_start`, `children_count`) pair into a contiguous `children_arena` shared across the whole forest, own logical and allocated sizes, and post-order-computed `value_logical`, `value_allocated`, and `item_count` aggregates. The path string is not stored on the node; it is lazy-computed by walking the parent chain when `apfs_scan_node_path` first asks for it, then cached on the `ApfsScan`. Random-access reads by index are constant-time; consumers need the indexed shape for the breadcrumb, the tree-list panel, the squarify recursion, and the hit-test loop.

INODE. Inode record. The FS-tree leaf that carries file/dir/symlink identity, mode, parent ID, link count, internal flags, and an xfield tail.

Logical size. User-visible byte length. The primary size metric, equal to POSIX `st_size` for regular files and to the symlink target byte length for symlinks. Distinct from allocated, exclusive, shared, and snapshot-retained sizes.

Max XID. The XID argument to an OMAP lookup. Returns the entry with the highest stored XID less than or equal to the requested value. The whole point of pinning the scan XID.

NamespaceEntry. One row of the namespace output: path, entry kind, file ID, logical size, optional allocated size, optional symlink target. Stable across raw and fallback backends.

Native shell. The SwiftUI macOS app whose visualisation is laid out by Rust and drawn by Core Graphics. The Rust crate is linked into the app process as a static library; every call from Swift into the indexer crosses a `cbindgen`-generated C ABI. No subprocess, no JSON serialisation, no WebKit. Chapter 13 documents the architecture; the repository still ships `viz/index.html` as a standalone drop-a-file-into-browser surface.

NX / NXSB. APFS container superblock (`nx_superblock_t`). The block-zero copy is the locator; the authoritative selected NXSB lives in the descriptor area at the highest valid checkpoint XID.

OID. Object identifier. A virtual address into an APFS OMAP’s name space. Not by itself a safe cache key (chapter 10).

OMAP. Object map. The B-tree that resolves virtual OIDs to physical addresses under a chosen scan XID. Container and volume each have their own.

Paddr. Physical block address. The on-disk location of an APFS object. Returned by an OMAP lookup for virtual objects; equal to `o_oid` for physical objects.

Parallel walker. The default fallback scheduler. Per-worker `BulkReader`, shared work queue, per-worker entry vectors merged on the main thread. Default thread count `min(hw.physicalcpu, 4)`, clamped to `[1, 4]`. The ceiling stays clear of the APFS container-lock regime that fires at $T > 4$ on Apple silicon.

Parsable. Source property: checkpoint and root discovery succeed; raw traversal runs. Necessary but not sufficient for “supported.”

--progress. CLI flag emitting stderr progress events at a 250 ms cadence. The parallel walker delivers events through a dedicated sampling thread so the cadence matches the serial path. The same stream is exposed on the FFI via `apfs_scan_directory_with_progress`, whose `ApfsProgressEvent` adds a cumulative logical-bytes field the SwiftUI shell uses as a determinate progress denominator.

Raw mode. The native APFS-parsing backend. Used when the source is in the raw-mode allowlist (detached `.dmg`, caller-supplied raw device, explicitly stable APFS source).

Readable. Source property: raw bytes can be opened and sampled. Cheap. Not, on its own, evidence of correctness.

Scan XID. The transaction identifier the parser pins for the duration of a run. All object resolution is performed against this XID. Picked as the highest checksum-valid NXSB XID in the descriptor area.

Sibling link / sibling map. Hard-link records. `j_sibling_link_val_t` carries a hard-link’s sibling-group identifier and the linked name; `j_sibling_map_val_t` closes the mapping from sibling ID to inode.

Snapshot, APFS. A read-only view of a volume at a point in time, named in the volume’s snapshot meta tree. Sealed-system snapshots (`com.apple.os.update-*`) are out of scope. The `--snapshot <mountpoint>` flag accepts an already-mounted snapshot directory; the mount step itself is privileged (chapter 13).

Squarify. The treemap-layout algorithm of Bruls, Huijzen, and van Wijk (2000), in the variant `d3-hierarchy`’s `treemapSquarify` implements. Greedy row-packing on the shorter side of each rect; rows grow until the worst-case aspect ratio would worsen, then lay out by dice (horizontal) or slice (vertical) and recurse on the remaining strip. The Rust port preserves the `d3` expressions for `alpha` and `beta`, so the two implementations produce the same rectangles for the same input.

Sparse bytes. `INO_EXT_TYPE_SPARSE_BYTES`: the count of bytes the sparse-allocation layer has not committed to disk. An allocation hint, not a logical size. Treated as diagnostic in the logical-size precedence and as the fail-closed gate for the allocated-size rule on sparse files.

Supported. Source property: the source is in the product allowlist for the requested mode. Validation has been performed; results were correct; the source class belongs in production.

--threads N. CLI flag selecting the parallel walker’s worker count. Default `min(hw.physicalcpu, 4)`; `N = 1` forces serial mode; larger values are accepted but pay the APFS-container contention cost documented in chapter 12.

Validatable. Source property: parser output can be compared against a named oracle for the requested mode. A prerequisite of “supported” but not, by itself, sufficient.

Virtual object. An APFS object whose `obj_phys_t` storage flag indicates the object is resolvable through an OMAP. `o_oid` is independent of the block’s `paddr`; the header is validated against `omap_lookup.oid`.

walk_skips. The fallback walker’s per-entry skip log. Records `EACCES`, `EPERM`, `ENOENT`, `firmlink_overlay_source`, and `duplicate_dir_inode` reasons with the path and keeps the walk going.

XID. Transaction identifier. Monotonic per container. Used to order checkpoints (chapter 3) and to disambiguate OMAP entries (chapter 4).

Xfield. The variable-length tail of an FS-tree record that carries extended fields. One source-backed cursor rule (chapter 6); fail-closed on any malformed shape (chapter 7).

Appendix: Experiment Register

A reference index of the controlled probes the manual cites. Each entry names the question the probe answered and the verdict it produced. Full READMEs, fixture-build scripts, and saved artifacts live under `docs/research/experiments/EX-*/`.

Probe	Question and result	Verdict
EX-01	Does the latest visible NXSB on a mounted lab image stay stable under concurrent writes? Found that it does not; latest-state raw reads do not have a stable oracle on live mounted volumes.	live-state-non-stable
EX-02	What minimum FS-record taxonomy reproduces the v1 namespace plus logical-size oracle? Identified DIR_REC, INODE, dstream fields, XATTR, SIBLING_LINK, SIBLING_MAP as the v1 record families.	required-record-set
EX-03	On a detached APFS image, does the raw walker reproduce the mounted POSIX namespace plus logical size? First positive proof loop.	pinned-state-parity
EX-04	Does the EX-03 parity hold across case-insensitive and case-sensitive APFS variants? Expanded fixture corpus; parity held.	expanded-corpus-parity
EX-05	On a still-mounted lab image under write churn, does latest-state raw output match any baseline or final mounted oracle? Counterexample result: it does not. Live latest cannot be a v1 model.	live-latest-not-validatable
EX-06	Across mutations to a tracked subtree, which identity fields stay stable? OID stays stable; paddr, XID, checksum, and block hash all change. Bare-OID cache is unsafe (chapter 10).	oid-alone-unsafe
EX-07	Under exact node-identity matches across the full nine-field key, does subtree-summary reuse ever produce a wrong summary? First run found zero false reuse in the detached lab corpus. Cache idea remains alive but unproven for production.	zero-false-reuse-on-corpus
EX-08	What does the read-path support matrix look like on a safe host? Detached unencrypted image-backed APFS is supported for narrow v1. Mounted images are readable but not supported. Startup-disk raw read is blocked by privilege.	read-path-matrix-baseline
EX-09	Which size metrics belong in v1, and which require their own oracle? Defined the accounting register: logical size is v1; allocated, exclusive, shared, snapshot-retained are deferred.	accounting-register

Probe	Question and result	Verdict
EX-10	Does the native Rust checkpoint scanner reach selected NXSB, checkpoint-map, container OMAP, volume superblock decode, volume OMAP, and FS-tree root validation on the proof fixture? Yes.	native-checkpoint-scanner
EX-11	Does the checkpoint-map ring validate end-to-end with CHECKPOINT_MAP_LAST enforced? Yes, on the proof fixture and on synthetic malformed cases.	checkpoint-map-integrity
EX-12	Does the native OMAP resolver satisfy SR-006 lower-bound (oid, max_xid) semantics, with hard stops at the encryption and unknown-flag cases? Yes; cross-validated against go-apfs identitydump.	validated-omap-lookup-contract
EX-13	Does a Python raw-byte body decoder reconstruct the mounted namespace and logical-size oracle on the proof fixture? Yes, modulo candidate-scored xfield layout (later resolved by SR-015 / EX-16).	validated-native-record-body-contract
EX-14	Does a second, denser xfield-bearing fixture variant resolve the xfield layout question? Returned oracle_inconclusive: Rust aborted with a checksum mismatch at block 1031 before xfield comparison could run. Blocker tracked into EX-15.	oracle-inconclusive
EX-15	What does block 1031 actually contain, and which of (a) stale- checkpoint selection, (b) missing checkpoint-map rule, (c) Rust validation bug, or (d) malformed source is the cause? Hypothesis (c). FS-tree internal-node 8-byte values are virtual OIDs requiring volume-OMAP resolution, not paddrs. Fix landed; two regression tests added.	validated-fs-tree-internal-oid-resolution-gap
EX-16	Does the SR-015 single xfield cursor rule decode every required xfield in the proof fixture with <code>xf_used_data == sum(round_up(x_size, 8))</code> and oracle parity? Yes, 14 of 14 records.	validated-sr-015-cursor-rule
EX-17	Does the Rust body decoder hard-stop on every SR-016 per-record malformed case with a typed <code>InvalidObject</code> ? Yes; 21 unit tests, all passing.	validated-sr-016-fail-closed-unit-tests
EX-18	Does the Rust body decoder produce field-level identical output to the Python decoder on the proof fixture? Yes: 53 records, identical (<code>node_paddr</code> , <code>entry_index</code>) set, zero divergent fields.	field-level-parity
EX-19	Does the SR-017 per-inode logical-size precedence reproduce <code>st_size</code> for ordinary, sparse, clone, hard-linked, symlink, and compressed cases? Yes, 5 of 5 unique inodes; the compressed case requires <code>INODE_HAS_UNCOMPRESSED_SIZE</code> to beat the decmpfs header.	validated-sr-017-precedence
EX-20	Do Rust paths match POSIX paths byte-for-byte on both case- insensitive and case-sensitive APFS volumes? Yes; volume case and normalisation flags propagate correctly. Lookup-by-name still not claimed.	validated-sr-018-name-preservation

Probe	Question and result	Verdict
EX-21	Does a POSIX-traversal fallback emit the same <code>NamespaceEntry</code> / <code>DirectoryAggregate</code> shape as Rust raw mode on the proof fixture? Yes; Python and Rust ports both pass.	validated-fallback-skeleton
EX-22	Does the SR-019 per-inode allocated-size precedence reproduce <code>st_blocks * 512</code> for ordinary, sparse, clone, hard-link, symlink, and decmpfs cases? Three of five emit-rows match; sparse overstates by exactly the sparse-bytes hint; decmpfs is correctly fail-closed. Sparse case split into a dedicated fail-closed branch pending an EX-22b sparse corpus.	partial-validated-sr-019-alloced-size
EX-23	Does the existing fallback walker produce byte-identical rows when pointed at a live volume and a read-only snapshot of the same volume? Probe ran on a clean dev workstation, found only the sealed-system OS-update snapshot (correctly skipped), and emitted a blocked verdict with the privileged-rerun reproducer. The shape claim itself remains unclaimed pending the rerun.	blocked-no-snapshots-at-all
EX-24	For the production walker's <code>getattrlistbulk</code> attribute mask, is the SR-021 vnode-rage hypothesis a measurable lever? Drec-only and current-mask landed within 1% on every dimension; <code>fts(3)</code> was $1.7\times$ slower than the current mask. The vnode-rage cost is not load-bearing; the Phase-2 deferred-attribute refactor is de-prioritised; <code>fts</code> follow-up cancelled.	vnode-cost-marginal
EX-25	What is the parallel walker's scaling envelope on Apple silicon at $T \in \{1, 2, 4, 8, 14\}$? $T=4$ reaches $2.47\times$ $T=1$; $T=8$ regresses; $T=14$ catastrophically loses ($9.3\times$ $T=1$ system CPU for $1.38\times$ throughput). The Szorc-2018 / Apple-DTS-2025 container-lock contention regime is real and starts at $T = 2 \times$ optimum on this kernel. Production default <code>min(hw.physicalcpu, 4)</code> .	validated-parallel-scaling
EX-26	Against the post-FFI workload (<code>/Users/kai/Projects</code> , 320k entries, $T=4$, warm cache), how much of the post-rewrite budget is structural rather than kernel-bound, and which structural rounds clear which fractions of it? Eight rounds landed against <code>TreeNode</code> , <code>NamespaceEntry</code> , <code>Tree::build</code> , <code>build_aggregates</code> , <code>VisitedDirs</code> , and the CLI-only aggregates <code>Vec</code> ; net -32% scan wall, -50% tree wall, -28% peak RSS, allocator pressure roughly halved (chapter 12 § Structural-Density Pass). Two candidates falsified in pass: <code>getattrlistbulk</code> buffer sizes 32 KiB through 2 MiB indistinguishable (4 MiB regresses), and the EX-24 vnode-rage hypothesis re-confirmed at the user-space layer. The bench artefacts (<code>examples/perf_probe</code> , <code>APFS_PHASE_TIMINGS</code> , <code>APFS_BULK_BUF_KIB</code>) remain in-tree for re-ablation.	validated-structural-density-pass